

Implémentation et évaluation de StreamKM++ sur Apache Spark

Jules MAURICE
Asma BEN-ZINE
Victorien GBAGUIDI

Août 2026

Table des matières

1	Introduction	2
2	Algorithmes	3
2.1	Algorithme de Lloyd pondéré	3
2.2	Initialisation k-means++	4
2.3	Construction du coreset	4
2.4	Merge-and-reduce	5
3	Apache Spark	7
3.1	RDD et DataFrames	7
3.2	Traitement par micro-batches avec Structured Streaming	7
3.3	Diffusion des centres avec broadcast	8
3.4	Réduction arborescente	8
3.5	Partitions et ressources de calcul	8
4	Implémentation	10
4.1	Construction distribuée du coreset	10
4.1.1	Construction d'un coreset par partition	10
4.1.2	Fusion des résumés locaux	11
4.2	Traitement avec Structured Streaming	11
4.3	Correspondance avec l'implémentation Flink	12
4.4	Évaluation distribuée du coût	12
4.5	Périmètre de l'implémentation	13
4.6	Synthèse du portage de Flink vers Spark	14
5	Évaluation expérimentale	15
5.1	Protocole expérimental	15
5.2	Qualité du clustering sur Coverttype	16
5.2.1	Influence du nombre de clusters	16
5.2.2	Comparaison avec Spark MLlib	17
5.2.3	Influence de la taille du coreset	17
5.2.4	Limites de l'évaluation	18
5.3	Optimisation du noyau local	18
5.3.1	Représentations étudiées	18
5.3.2	Protocole JMH	18
5.3.3	Résultats	19
5.4	Synthèse	19
6	Discussion	21
6.1	Apports de l'implémentation	21
6.2	Limites de l'implémentation	22
6.3	Bilan	24

7	Conclusion et perspectives	25
7.1	Perspectives	25
A	Code source complet	27
A.0.1	Configuration du projet	27
A.0.2	Couche core : structures de données et k-means	28
A.0.3	Couche coreset : coreset tree et merge-and-reduce	34
A.0.4	Couche spark : portage distribué	42
A.0.5	Couche experiments : harnais de mesure de qualité	46
A.0.6	Fonction Main	62
A.0.7	Suite de tests	64

Chapitre 1

Introduction

Le clustering de grands volumes de données présente des contraintes particulières lorsque les observations arrivent sous forme de flux. Contrairement à une approche batch classique, il n'est pas toujours possible de conserver l'ensemble des données en mémoire ni de les parcourir plusieurs fois. Les algorithmes de clustering adaptés aux flux doivent donc limiter leur consommation mémoire tout en maintenant une qualité de partitionnement satisfaisante.

StreamKM++, proposé par Ackermann et al. (2012) [2], répond à cette problématique en construisant progressivement un *coreset*, c'est-à-dire un résumé pondéré et de taille limitée des données observées. Le clustering final est ensuite réalisé sur ce résumé à l'aide de k-means++ [3]. Cette approche permet de réduire la quantité de données manipulée lors de l'étape finale de clustering tout en conservant une représentation des données adaptée au calcul des centres.

Bitsakis (2018) [1] a proposé une implémentation distribuée de StreamKM++ avec Apache Flink. Le présent projet étudie l'adaptation de cette approche à Apache Spark, notamment à travers les RDD et Structured Streaming. Il ne s'agit pas uniquement de transposer l'implémentation existante, mais de l'adapter aux mécanismes propres à Spark, tels que le partitionnement des données, les opérations de réduction distribuée et le traitement par micro-batches.

Le travail s'articule autour de trois objectifs principaux. Le premier consiste à implémenter et à valider les composants algorithmiques nécessaires à StreamKM++, notamment l'algorithme de Lloyd pondéré, l'initialisation k-means++, la construction du coreset et la stratégie *merge-and-reduce*. Le deuxième concerne leur adaptation à une exécution distribuée avec Spark, dans laquelle chaque partition construit un résumé local avant la fusion des résultats. Enfin, l'implémentation est évaluée expérimentalement afin d'étudier la qualité du clustering obtenu et l'influence de la taille du coreset.

L'architecture retenue distingue la logique algorithmique de l'orchestration distribuée. Les packages `streamkm.core` et `streamkm.coreset` regroupent les composants indépendants de Spark, tandis que `streamkm.spark` prend en charge leur exécution distribuée. Cette séparation permet de valider les composants algorithmiques indépendamment de leur distribution avant d'évaluer la qualité des résultats produits dans l'environnement Spark.

Chapitre 2

Algorithmes

StreamKM++ est un algorithme de clustering conçu pour traiter des flux de données dont la taille peut rendre impossible la conservation de l'ensemble des observations en mémoire. Son principe consiste à construire progressivement un *coreset*, c'est-à-dire un résumé pondéré et de taille limitée des données déjà observées. Le clustering final est ensuite effectué sur ce résumé plutôt que sur l'ensemble du flux [2].

L'approche repose sur plusieurs composants complémentaires : une version pondérée de l'algorithme de Lloyd, l'initialisation k-means++, la construction d'un coreset à l'aide d'un arbre de partitionnement et une stratégie *merge-and-reduce* permettant de maintenir ce résumé au fur et à mesure de l'arrivée des données. Ces composants sont présentés dans les sections suivantes, en précisant les choix effectués dans l'implémentation lorsqu'ils diffèrent de la formulation de référence.

2.1 Algorithme de Lloyd pondéré

L'algorithme de Lloyd constitue l'étape d'optimisation utilisée par k-means. À partir d'un ensemble de centres initiaux, il alterne deux opérations jusqu'à convergence. Chaque point est d'abord affecté au centre le plus proche, puis chaque centre est recalculé à partir des points qui lui sont associés.

Dans StreamKM++, cette opération est appliquée au coreset plutôt qu'aux observations originales. Les représentants du coreset possèdent un poids correspondant à la quantité de données qu'ils représentent. Ces poids doivent donc être pris en compte lors du recalcul des centres.

Soit un ensemble de points pondérés

$$P = \{(p_i, w_i)\}_{i=1}^n$$

et un ensemble de centres

$$C = \{c_1, \dots, c_k\}.$$

Après l'affectation de chaque point à son centre le plus proche, le centre du cluster j est recalculé selon :

$$c_j = \frac{\sum_{p_i \in C_j} w_i p_i}{\sum_{p_i \in C_j} w_i}.$$

La qualité du clustering est évaluée à l'aide de la somme pondérée des distances euclidiennes au carré :

$$\text{cost}(P, C) = \sum_{i=1}^n w_i \min_{c \in C} \|p_i - c\|^2.$$

La pondération permet à un représentant de poids w d'avoir le même effet sur le calcul des centres que w observations identiques de poids unitaire. Elle est donc indispensable lorsqu'un point du coreset représente plusieurs observations du jeu de données initial.

Dans l'implémentation réalisée, cette propriété est vérifiée par des tests unitaires. Lorsqu'un cluster devient vide au cours d'une itération, son centre précédent est conservé afin d'éviter une réinitialisation supplémentaire. L'algorithme s'arrête lorsque l'amélioration relative du coût devient inférieure à un seuil fixé ou lorsque le nombre maximal d'itérations est atteint.

2.2 Initialisation k-means++

L'algorithme de Lloyd est sensible au choix des centres initiaux. Une sélection entièrement aléatoire peut conduire à des centres concentrés dans une même région des données et ainsi dégrader la qualité du résultat obtenu.

L'initialisation k-means++ proposée par Arthur et Vassilvitskii [3] cherche à mieux répartir les centres initiaux. Après la sélection d'un premier centre, chaque nouveau centre est choisi avec une probabilité proportionnelle au carré de sa distance au centre déjà sélectionné le plus proche.

Pour un point p , on définit :

$$D(p)^2 = \min_{c \in C} \|p - c\|^2.$$

Dans le cas pondéré utilisé avec le coreset, la probabilité de sélectionner un point p_i comme nouveau centre est proportionnelle à :

$$w_i D(p_i)^2.$$

Ainsi, les représentants ayant un poids important et situés loin des centres déjà sélectionnés ont une probabilité plus élevée d'être choisis.

Choix du premier centre. Dans la formulation classique de k-means++, le premier centre est sélectionné uniformément parmi les observations [3]. Dans l'implémentation réalisée, lorsque les données sont pondérées, ce premier centre est choisi avec une probabilité proportionnelle au poids du point. Les deux approches sont équivalentes lorsque tous les poids valent 1. Dans le cas d'un coreset, ce choix permet de prendre en compte dès l'initialisation la masse de données représentée par chaque point.

Afin de limiter la sensibilité du résultat à l'initialisation, le processus composé de k-means++ puis de l'algorithme de Lloyd est exécuté plusieurs fois. Le modèle présentant le coût final le plus faible est conservé. Dans les expériences réalisées, cinq exécutions sont utilisées sauf indication contraire.

2.3 Construction du coreset

L'objectif principal de StreamKM++ est de réduire la quantité de données conservée tout en maintenant suffisamment d'information pour obtenir un clustering représentatif. Cette réduction repose sur la construction d'un coreset [2].

Un coreset est un ensemble pondéré de taille réduite représentant les principales caractéristiques des données originales. Chaque point du coreset est associé à un poids correspondant à la masse des observations qu'il résume.

Dans StreamKM++, ce résumé est construit à l'aide d'un *coreset tree*. Les données sont organisées progressivement dans un arbre binaire dont chaque feuille correspond à un sous-ensemble des observations et possède un représentant.

Initialement, une seule feuille contient l'ensemble des points à résumer. À chaque étape, une feuille est sélectionnée pour être subdivisée. La probabilité de sélection dépend de son coût :

une région dont les données sont mal représentées possède un coût plus important et est donc davantage susceptible d'être raffinée.

Une fois la feuille sélectionnée, un nouveau représentant est choisi à partir des distances au représentant existant. Les observations sont ensuite réparties entre les deux représentants selon leur proximité. Ce processus est répété jusqu'à atteindre la taille cible du coreset ou jusqu'à ce qu'aucune subdivision supplémentaire ne soit possible.

Coût et masse. Deux grandeurs doivent être distinguées lors de la construction de l'arbre. Le coût d'un nœud est calculé à partir des distances entre les observations qu'il contient et son représentant. Il intervient dans le choix des régions à subdiviser.

La masse d'un nœud correspond quant à elle à la somme des poids des observations qu'il représente. Elle devient le poids du représentant lorsque celui-ci est intégré au coreset.

Dans l'implémentation réalisée, le coût et la masse sont donc représentés séparément. Le premier intervient dans la construction de l'arbre, tandis que la seconde est utilisée pour pondérer les représentants produits.

Conservation de la masse. La construction du coreset doit préserver la masse totale des données. Si l'ensemble d'entrée possède une masse

$$W = \sum_i w_i,$$

alors le coreset S doit vérifier :

$$\sum_{q \in S} w_q = W.$$

La réduction du nombre de représentants ne modifie donc pas la quantité totale de données représentée. Cette propriété constitue l'un des invariants vérifiés par les tests de l'implémentation.

Cas dégénérés. La valeur m définit une taille cible maximale pour le coreset. Dans le cas général, la construction cherche à produire m représentants. Certaines configurations peuvent toutefois empêcher une nouvelle subdivision, notamment lorsque plusieurs observations sont identiques et que le coût d'une feuille devient nul.

Dans cette situation, le coreset peut contenir moins de m représentants sans que cela constitue une erreur. Sa taille reste néanmoins bornée par m et la masse totale des données demeure conservée.

2.4 Merge-and-reduce

Le *coreset tree* permet de résumer un ensemble fini de points. Pour traiter progressivement un flux dont la taille totale n'est pas nécessairement connue à l'avance, StreamKM++ utilise une stratégie *merge-and-reduce* [2].

Le principe consiste à maintenir plusieurs niveaux de stockage, appelés buckets :

$$B_0, B_1, B_2, \dots$$

Le premier niveau reçoit les observations du flux. Lorsque la quantité de données à traiter atteint le seuil défini par la taille du coreset, les données sont résumées puis propagées vers les niveaux supérieurs.

Si le niveau suivant est libre, le résumé y est stocké. Lorsqu'il contient déjà un coreset, les deux résumés sont réunis puis réduits à nouveau afin de produire un nouveau coreset de taille bornée. Cette propagation se poursuit tant que le niveau suivant est occupé.

Le fonctionnement est comparable à la propagation d'une retenue dans une représentation binaire. Les résumés sont progressivement fusionnés afin de représenter des portions de plus en plus importantes du flux sans conserver l'ensemble des observations originales.

Invariants de la structure. Les résumés stockés dans les différents niveaux respectent deux propriétés principales :

- leur taille reste bornée par m ;
- la masse totale représentée est conservée lors des différentes fusions.

La taille d'un résumé peut être inférieure à m dans les cas dégénérés décrits précédemment. L'invariant porte donc sur une taille maximale plutôt que sur l'obtention systématique d'exactly m représentants.

Coreset courant. À tout instant, l'ensemble des buckets non vides constitue un résumé pondéré des observations déjà traitées. Le nombre de niveaux augmente avec la quantité de données reçue, ce qui permet de contrôler la consommation mémoire sans nécessiter la connaissance préalable de la taille totale du flux.

Lorsqu'un clustering est demandé, les représentants contenus dans les différents buckets sont regroupés afin de constituer le coreset courant. L'initialisation k-means++ et l'algorithme de Lloyd pondéré sont ensuite appliqués à ce résumé.

Fusion de résumés indépendants. La composition des coresets permet également de combiner des résumés construits indépendamment sur des sous-ensembles distincts des données. Les représentants issus de plusieurs résumés peuvent être réunis puis réduits à nouveau afin d'obtenir un coreset représentant leur union.

Cette propriété est particulièrement adaptée à l'exécution distribuée avec Spark. Chaque partition peut construire indépendamment son propre résumé à l'aide d'un `BucketManager`. Les coresets obtenus sont ensuite fusionnés progressivement afin de produire un résumé global.

La logique de StreamKM++ reste ainsi concentrée dans les packages `streamkm.core` et `streamkm.coreset`. La couche `streamkm.spark` prend en charge la distribution des traitements sans modifier le fonctionnement interne de la construction du coreset.

Chapitre 3

Apache Spark

La thèse de référence présente le modèle d'exécution d'Apache Flink avant de détailler l'implémentation distribuée de StreamKM++. Une démarche similaire est adoptée ici avec Apache Spark. Ce chapitre se concentre uniquement sur les mécanismes utilisés dans le projet : les RDD et les DataFrames, le traitement par micro-batches avec Structured Streaming, les variables diffusées, la réduction arborescente et le partitionnement des données.

L'objectif n'est donc pas de proposer une présentation générale de Spark, mais d'identifier les mécanismes qui interviennent directement dans l'adaptation de StreamKM++. Leur mise en œuvre dans le projet est présentée au chapitre 4.

3.1 RDD et DataFrames

Apache Spark fournit plusieurs abstractions pour le traitement distribué des données. Deux d'entre elles sont utilisées dans l'implémentation : les *Resilient Distributed Datasets* (RDD) et les DataFrames.

Un RDD est une collection de données distribuée et partitionnée. Les transformations telles que `map`, `filter` ou `mapPartitions` peuvent être appliquées indépendamment aux différentes partitions. Cette abstraction est adaptée aux objets `Point` manipulés par StreamKM++, dont le traitement repose notamment sur le calcul de distances, la construction de coresets et la mise à jour des structures de *merge-and-reduce*.

Les DataFrames représentent quant à eux des données distribuées organisées selon un schéma. Ils permettent à Spark d'appliquer ses mécanismes d'optimisation aux opérations structurées telles que les projections, les filtres et les agrégations.

Dans l'architecture retenue, les DataFrames sont principalement utilisés pour l'entrée des données et l'intégration avec Structured Streaming. La logique algorithmique de StreamKM++ est ensuite exécutée à partir de RDD contenant des objets `Point`. Cette séparation permet de conserver une logique algorithmique distincte des mécanismes d'orchestration propres à Spark.

3.2 Traitement par micro-batches avec Structured Streaming

Structured Streaming permet de traiter des flux de données en s'appuyant sur les mécanismes d'exécution de Spark. Dans le cadre de ce projet, le traitement repose sur le mode *micro-batch*.

Le flux d'entrée est découpé en lots successifs, chacun étant traité comme un ensemble de données borné. La méthode `foreachBatch` permet d'appliquer un traitement spécifique à chaque micro-batch produit par le moteur.

Ce fonctionnement est différent du modèle de streaming utilisé dans l'implémentation Flink étudiée par Bitsakis. Avec Structured Streaming, chaque micro-batch constitue directement une unité de traitement identifiable. Le lot peut ainsi être traité de manière distribuée avant que le résumé obtenu soit intégré à l'état maintenu par l'application.

Cette approche implique toutefois que les observations sont traitées par lots et non individuellement dès leur arrivée. Le traitement dépend donc de la fréquence de déclenchement des micro-batches. Les conséquences de ce choix sont discutées au chapitre 6.

3.3 Diffusion des centres avec broadcast

Certaines opérations nécessitent qu'une même information soit accessible aux différentes tâches Spark. C'est notamment le cas lors de l'évaluation du clustering, où les centres calculés doivent être utilisés par toutes les partitions contenant les observations.

Spark fournit pour cela des variables diffusées, ou *broadcast variables*. Une valeur diffusée est transmise aux exécuteurs afin d'être réutilisée localement par les tâches qui en ont besoin. Ce mécanisme évite de transférer plusieurs fois la même donnée avec chaque tâche.

Dans l'implémentation, ce mécanisme est utilisé par `CostEvaluator.cost`. Les centres du clustering sont diffusés aux exécuteurs, puis chaque partition calcule localement sa contribution à la somme des erreurs quadratiques. Les contributions partielles sont ensuite agrégées afin d'obtenir le coût global.

La mise en œuvre de cette évaluation est détaillée dans la section 4.4.

3.4 Réduction arborescente

La construction distribuée de StreamKM++ produit plusieurs résumés intermédiaires, généralement un par partition. Ces résumés doivent ensuite être combinés afin de construire un coreset global.

Une approche simple consisterait à récupérer l'ensemble des résumés sur le driver puis à effectuer les fusions séquentiellement. Cette organisation centraliserait toutefois l'ensemble du travail de réduction sur un seul processus.

Spark fournit des opérations de réduction arborescente telles que `treeReduce` et `treeAggregate`. Les résultats intermédiaires sont alors combinés progressivement à plusieurs niveaux avant l'obtention du résultat final. La fusion peut ainsi être partiellement distribuée au lieu d'être réalisée entièrement sur le driver.

Ce principe est adapté à StreamKM++, puisque deux résumés peuvent être fusionnés puis réduits afin de produire un nouveau coreset pouvant lui-même être combiné avec un autre résumé.

Dans l'implémentation réalisée, `treeReduce` est utilisé pour fusionner les structures `BucketManager` construites sur les différentes partitions. Cette organisation permet de conserver une construction locale indépendante pour chaque partition avant la production du résumé global.

La réduction arborescente permet ainsi d'éviter une centralisation immédiate de toutes les opérations de fusion. Son influence sur les performances est étudiée expérimentalement dans le chapitre consacré à l'évaluation.

3.5 Partitions et ressources de calcul

Le partitionnement constitue un élément central du modèle d'exécution de Spark. Un RDD est divisé en plusieurs partitions, chacune pouvant être traitée par une tâche indépendante.

Dans l'implémentation de StreamKM++, chaque partition construit son propre `BucketManager` et produit un résumé local des observations qu'elle contient. Le nombre de partitions détermine donc le nombre de constructions locales et influence la manière dont les différents coresets sont ensuite fusionnés.

Le nombre de partitions doit être distingué des ressources de calcul disponibles. Le premier détermine la décomposition logique des données et le nombre de tâches susceptibles d'être exécutées, tandis que le nombre de cœurs disponibles détermine le nombre de tâches pouvant effectivement être exécutées simultanément.

Une augmentation du nombre de partitions n'entraîne donc pas nécessairement une amélioration des performances. Un partitionnement plus fin peut accroître les possibilités de parallélisme, mais également augmenter le nombre de tâches ainsi que les coûts liés à leur ordonnancement, à la sérialisation des données et aux opérations de fusion.

Le partitionnement peut également influencer le coresets produit. Chaque partition réalise en effet une construction locale indépendante avant la fusion des résultats. Modifier le découpage des données peut ainsi conduire à des résumés intermédiaires différents et, par conséquent, à un coresets global légèrement différent.

Cette distinction est importante pour comprendre le comportement de l'implémentation distribuée et l'influence du partitionnement sur la construction des coresets.

Chapitre 4

Implémentation

Ce chapitre présente l'implémentation distribuée de StreamKM++ avec Apache Spark. La logique algorithmique décrite au chapitre 2 reste indépendante de Spark et est regroupée dans les packages `streamkm.core` et `streamkm.coreset`. La couche `streamkm.spark` prend en charge la distribution des données, la construction des coreset locaux, leur fusion et l'évaluation du clustering obtenu.

L'implémentation repose sur deux niveaux de traitement. Chaque partition construit d'abord un résumé local des données qui lui sont attribuées. Ces résumés sont ensuite fusionnés afin d'obtenir un coreset global sur lequel le clustering final est calculé. Structured Streaming reprend ce même principe pour traiter successivement plusieurs micro-batches.

4.1 Construction distribuée du coreset

4.1.1 Construction d'un coreset par partition

La première étape consiste à répartir la construction du coreset entre les partitions du RDD. La méthode `StreamKMPlusPlus.partialCoresets` utilise `mapPartitionsWithIndex` afin que chaque partition construise indépendamment son propre résumé.

```
def partialCoresets(
  rdd: RDD[Point],
  p: StreamKMParams
): RDD[Array[Point]] = {

  rdd.mapPartitionsWithIndex { (idx, it) =>
    val bm = new BucketManager(p.m, p.seed + idx)
    bm.insertAll(it)
    Iterator.single(bm.extractCoreset())
  }
}
```

Chaque partition instancie un `BucketManager`, lui transmet les observations qu'elle contient puis extrait le coreset obtenu. La graine utilisée est dérivée de l'indice de la partition afin de disposer d'une construction locale reproductible pour un partitionnement donné.

Cette organisation correspond naturellement au modèle distribué de Spark. Les partitions peuvent être traitées indépendamment avant la phase de fusion, ce qui évite de centraliser la construction du coreset sur le driver.

4.1.2 Fusion des résumés locaux

Les coresets construits sur les différentes partitions doivent ensuite être combinés afin de produire un résumé global. La méthode `mergeCoresets` applique pour cela une réduction arborescente.

```
def mergeCoresets(
  rdd: RDD[Point],
  p: StreamKMPParams,
  depth: Int = 2
): Array[Point] = {

  val perPartition: RDD[BucketManager] =
    rdd.mapPartitionsWithIndex { (idx, it) =>
      val bm = new BucketManager(p.m, p.seed + idx)
      bm.insertAll(it)
      Iterator.single(bm)
    }

  perPartition
    .treeReduce((a, b) => a.merge(b), depth)
    .extractCoreset()
}
```

Chaque partition produit ici un `BucketManager` contenant son résumé local. Les structures obtenues sont ensuite combinées à l'aide de `treeReduce`. L'opération de fusion repose directement sur la méthode `BucketManager.merge`, déjà utilisée par la stratégie *merge-and-reduce* décrite au chapitre 2.

Cette approche permet de réutiliser la même logique de fusion indépendamment du niveau auquel elle est appliquée. La couche Spark organise ainsi l'exécution distribuée sans modifier les mécanismes internes de construction du coreset.

Par rapport à une récupération de l'ensemble des résumés sur le driver suivie d'une fusion séquentielle, la réduction arborescente répartit une partie des opérations de combinaison entre plusieurs niveaux.

4.2 Traitement avec Structured Streaming

La méthode `StreamKMPlusPlus.run` étend la construction distribuée au traitement d'un flux avec Structured Streaming. Un `BucketManager` global est maintenu sur le driver afin de conserver un résumé des micro-batches déjà traités.

```
def run(
  source: DataFrame,
  p: StreamKMPParams,
  onModel: KMeansModel => Unit,
  featuresCol: String = "features"
): StreamingQuery = {

  val global = new BucketManager(p.m, p.seed)

  source.writeStream.foreachBatch { (batch, batchId) =>
```

```

val points = batch.rdd.map { row =>
  Point(
    row.getAs[Seq[Double]](featuresCol).toArray
  )
}

val partial =
  mergeCoresets(
    points,
    p.copy(seed = p.seed + batchId)
  )

partial.foreach(global.insert)

val model =
  KMeans.fit(
    global.extractCoreset(),
    p.k,
    p.nRestarts,
    seed = p.seed
  )

onModel(model)

}.start()
}

```

Chaque micro-batch est d'abord converti en RDD de **Point**. Un coreset est ensuite construit de manière distribuée à partir des partitions du lot. Le résumé obtenu est intégré au **BucketManager** global, qui représente progressivement l'ensemble des observations déjà traitées.

Le clustering est calculé à partir du coreset courant après l'intégration du micro-batch. La méthode **onModel** permet ensuite d'exploiter le modèle produit sans imposer un mécanisme de sortie particulier.

Les observations brutes sont considérées comme ayant un poids unitaire. Les poids supérieurs à un apparaissent uniquement lors de la construction des coreset et correspondent à la masse représentée par chaque point du résumé.

4.3 Correspondance avec l'implémentation Flink

L'implémentation Spark conserve les principales étapes de l'approche distribuée proposée avec Flink par Bitsakis [1], mais leur réalisation est adaptée au modèle d'exécution de Spark.

La principale différence concerne la fusion des résumés intermédiaires. Dans l'implémentation Spark, les coreset locaux peuvent être combinés au moyen d'une réduction arborescente. Le découpage en micro-batches fournit également une frontière explicite entre les différents lots traités.

4.4 Évaluation distribuée du coût

La qualité du clustering doit être évaluée sur l'ensemble des données originales et non uniquement sur le coreset. Le coût utilisé dans les expériences correspond à la somme des distances euclidiennes au carré entre chaque observation et son centre le plus proche.

Opérateur Flink	Rôle	Implémentation Spark
HdfsSource	Lecture distribuée des données	Source Spark partitionnée
HandleWatermark	Identification d'une frontière de traitement	Frontière fournie par les micro-batches de Structured Streaming
FlatMapToPoint	Conversion des données en points	Transformation en objets Point
FlatMapToPartialCoreset	Construction locale du coreset	mapPartitionsWithIndex
FlatMapToFinalCoreset	Fusion des coreset partiels	treeReduce et BucketManager.merge
FlatMapToKmeansPP	Calcul du clustering final	KMeans.fit sur le coreset global
HdfsSink	Production des centres	Fonction de sortie onModel

TABLE 4.1 – Correspondance entre les principales étapes des implémentations Flink et Spark.

Cette opération est réalisée de manière distribuée par la méthode `CostEvaluator.cost` :

```
def cost(
  points: RDD[Point],
  centers: Array[Array[Double]],
  depth: Int = 2
): Double = {

  val bcCenters =
    points.sparkContext.broadcast(centers)

  points
    .mapPartitions { it =>
      Iterator.single(
        KMeans.cost(
          it.toArray,
          bcCenters.value
        )
      )
    }
    .treeReduce(_ + _, depth)
}
```

Les centres sont d'abord diffusés aux exécuteurs à l'aide d'une variable `broadcast`. Chaque partition calcule ensuite localement la contribution de ses observations au coût total. Les coûts partiels sont enfin additionnés à l'aide de `treeReduce`.

Cette méthode évite de rapatrier les observations sur le driver pour effectuer l'évaluation. Elle est utilisée dans les expériences du chapitre 5 afin de comparer les différentes configurations sur la même métrique et sur l'ensemble du jeu de données.

4.5 Périmètre de l'implémentation

La thèse de Bitsakis décrit également une variante permettant de produire périodiquement plusieurs clusterings pour différentes valeurs de k , ainsi qu'un mécanisme d'interrogation de l'état du traitement.

Ces fonctionnalités n’ont pas été retenues dans le périmètre de l’implémentation réalisée. Le travail se concentre sur la construction distribuée de StreamKM++, son adaptation à Structured Streaming et l’évaluation expérimentale de la qualité et des performances du portage vers Spark.

Structured Streaming permet néanmoins de recalculer un modèle après chaque micro-batch à partir du coreset courant, comme le montre la méthode `run`. Cette fonctionnalité fournit le mécanisme nécessaire à la production progressive de résultats sans ajouter de gestion spécifique de requêtes périodiques.

4.6 Synthèse du portage de Flink vers Spark

Le portage vers Spark conserve les principes algorithmiques de l’implémentation distribuée de StreamKM++, tout en adaptant leur orchestration au modèle d’exécution du framework.

La construction locale des coresets est naturellement associée aux partitions Spark. Les résumés produits peuvent ensuite être combinés à l’aide de `treeReduce`, en réutilisant directement les opérations de fusion définies dans `BucketManager`. Structured Streaming permet quant à lui de traiter successivement les différents micro-batches et de maintenir un coreset représentant les données déjà consommées.

L’évaluation du coût suit la même logique distribuée : les centres sont diffusés aux partitions et les contributions locales sont agrégées pour produire la somme des erreurs quadratiques sur l’ensemble des observations.

Cette architecture permet ainsi de conserver une séparation claire entre la logique algorithmique de StreamKM++ et les mécanismes de distribution propres à Spark. Les expérimentations présentées dans le chapitre suivant permettent d’évaluer la qualité des coresets obtenus ainsi que le comportement de cette implémentation lorsque la taille des données, le partitionnement et les ressources de calcul varient.

Chapitre 5

Évaluation expérimentale

Ce chapitre présente l'évaluation expérimentale de l'implémentation de StreamKM++. L'objectif principal est d'étudier la qualité du clustering obtenu avec la version distribuée et de la comparer à l'implémentation séquentielle ainsi qu'à Spark MLlib. L'influence du nombre de clusters et de la taille du coreset est étudiée sur le jeu de données Covertype. Une expérimentation complémentaire analyse également l'effet d'une représentation mémoire alternative sur les performances du noyau local.

5.1 Protocole expérimental

Les expériences de qualité sont réalisées sur le jeu de données Covertype. Celui-ci contient 581 012 observations décrites par 54 attributs numériques utilisés pour le clustering. La dernière colonne, correspondant à la classe associée à chaque observation, n'est pas utilisée puisque l'algorithme étudié repose sur un apprentissage non supervisé.

Pour les expériences distribuées, Apache Spark est configuré avec huit partitions. Chaque partition traite une partie des observations et construit un coreset local. Les résumés obtenus sont ensuite fusionnés afin de produire un coreset global sur lequel les centres finaux sont calculés.

Une graine fixe égale à 2026 est utilisée afin de rendre les différentes configurations reproductibles. Chaque configuration est exécutée une seule fois. Ce protocole permet de comparer les différentes approches dans des conditions identiques, mais ne permet pas de mesurer la variabilité associée aux étapes aléatoires de StreamKM++ et de l'initialisation k-means++.

Avant ces expériences, des tests unitaires et déterministes ont été réalisés sur les principaux composants de l'implémentation. Ils vérifient notamment le calcul des distances, le fonctionnement du k-means pondéré, la conservation de la masse lors de la construction et de la fusion des coresets ainsi que le chargement des 54 attributs utilisés pour Covertype.

La qualité du clustering est évaluée à l'aide de la somme des erreurs quadratiques, ou *Sum of Squared Errors* (SSE). Pour un ensemble de données X et un ensemble de centres C , cette mesure est définie par :

$$\text{SSE}(X, C) = \sum_{x \in X} \min_{c \in C} \|x - c\|^2.$$

Une valeur de SSE plus faible signifie que les observations sont globalement plus proches du centre auquel elles sont affectées.

Dans les expériences réalisées, le coreset est utilisé pour déterminer les centres, mais la SSE est ensuite calculée sur l'ensemble des observations originales. La comparaison porte ainsi sur la qualité du clustering obtenu sur le jeu de données complet et non uniquement sur son résumé.

Trois approches sont comparées :

- StreamKM++ séquentiel ;
- StreamKM++ distribué avec Apache Spark ;

— K-Means de Spark MLlib.

La version séquentielle constitue une référence interne permettant d’observer l’effet de la construction distribuée du coreset. Spark MLlib fournit une référence externe en appliquant K-Means directement aux données distribuées, sans construction préalable d’un coreset. La comparaison porte donc sur la qualité des centres obtenus et non sur une équivalence stricte entre les méthodes.

5.2 Qualité du clustering sur Covertypes

Deux paramètres sont étudiés : le nombre de clusters k et la taille maximale m du coreset. La première expérience analyse l’évolution de la SSE lorsque le nombre de centres augmente. La seconde étudie l’influence de la quantité d’information conservée dans le coreset.

5.2.1 Influence du nombre de clusters

La première expérience fait varier le nombre de clusters selon :

$$k \in \{10, 20, 30, 40, 50\}.$$

Pour chaque valeur de k , la taille du coreset est définie par :

$$m = 200k.$$

Le tableau 5.1 présente les résultats obtenus.

k	StreamKM++ séquentiel	StreamKM++ Spark	Spark MLlib
10	3.400×10^{11}	3.393×10^{11}	3.388×10^{11}
20	2.027×10^{11}	2.040×10^{11}	2.023×10^{11}
30	1.546×10^{11}	1.545×10^{11}	1.533×10^{11}
40	1.298×10^{11}	1.293×10^{11}	1.281×10^{11}
50	1.144×10^{11}	1.139×10^{11}	1.125×10^{11}

TABLE 5.1 – Influence du nombre de clusters sur la SSE pour les 581 012 observations de Covertypes, avec $m = 200k$.

Pour les trois méthodes, la SSE diminue nettement lorsque le nombre de clusters augmente. Elle passe d’environ 3.4×10^{11} pour $k = 10$ à environ 1.1×10^{11} pour $k = 50$.

Ce comportement est cohérent avec le fonctionnement de K-Means. Lorsque le nombre de centres augmente, les observations peuvent être représentées par des clusters plus fins, ce qui réduit leur distance au centre le plus proche.

Les résultats obtenus avec les versions séquentielle et distribuée de StreamKM++ sont très proches. Pour toutes les valeurs de k étudiées, l’écart relatif entre leurs SSE reste inférieur à 0.7 %. L’écart maximal est observé pour $k = 20$, avec environ 0.65 %.

Les différences observées peuvent s’expliquer par l’ordre de construction et de fusion des coresets ainsi que par les étapes aléatoires de l’algorithme. Elles restent néanmoins faibles au regard des coûts obtenus.

Dans les configurations testées, la distribution de la construction du coreset entre les partitions Spark ne conduit donc pas à une dégradation notable de la qualité du clustering par rapport à l’implémentation séquentielle.

5.2.2 Comparaison avec Spark MLlib

Les résultats du tableau 5.1 montrent également que les SSE produites par Spark MLlib restent proches de celles obtenues avec StreamKM++.

Pour toutes les valeurs de k testées, MLlib obtient une SSE légèrement inférieure à celle de StreamKM++ Spark. L'écart est d'environ 0.15 % pour $k = 10$, reste inférieur à 1 % pour $k = 20, 30$ et 40 , puis atteint environ 1.24 % pour $k = 50$.

Cette différence doit être interprétée en tenant compte du fonctionnement des deux approches. Spark MLlib applique K-Means directement à l'ensemble des observations distribuées, tandis que StreamKM++ construit d'abord un résumé pondéré de taille limitée puis détermine les centres à partir de ce résumé.

La construction du coreset introduit donc une approximation supplémentaire. Malgré cette réduction du jeu de données utilisé pour le calcul des centres, les SSE obtenues restent proches de celles de MLlib dans les configurations testées.

Ces résultats montrent ainsi que le coreset conserve suffisamment d'information pour produire un clustering dont la qualité reste proche de celle obtenue en appliquant directement K-Means aux données distribuées.

5.2.3 Influence de la taille du coreset

La deuxième expérience étudie l'influence de la taille du coreset. Le nombre de clusters est fixé à :

$$k = 25,$$

et les tailles suivantes sont testées :

$$m \in \{1000, 5000, 10000, 20000\}.$$

Le tableau 5.2 présente les résultats obtenus.

m	StreamKM++ séquentiel	StreamKM++ Spark
1 000	1.807×10^{11}	1.811×10^{11}
5 000	1.748×10^{11}	1.759×10^{11}
10 000	1.746×10^{11}	1.744×10^{11}
20 000	1.742×10^{11}	1.739×10^{11}

TABLE 5.2 – Influence de la taille du coreset pour $k = 25$ sur les 581 012 observations de Coverttype.

Dans cette expérience, la SSE diminue lorsque la taille du coreset augmente pour les deux implémentations.

Pour la version séquentielle, elle passe de 1.807×10^{11} avec $m = 1000$ à 1.742×10^{11} avec $m = 20000$. La version Spark présente une évolution similaire, avec une diminution de 1.811×10^{11} à 1.739×10^{11} .

La diminution la plus importante est observée entre $m = 1000$ et $m = 5000$. Pour les valeurs supérieures, le gain devient plus faible.

Ce comportement est cohérent avec le rôle du coreset. Une taille plus importante permet de conserver davantage d'information sur les données originales et peut ainsi améliorer la qualité de l'approximation. Les résultats montrent cependant que l'amélioration supplémentaire devient limitée lorsque la taille du résumé augmente.

Il apparaît ainsi un compromis entre la quantité d'information conservée et la taille du coreset manipulé lors du clustering final.

Les versions séquentielle et distribuée restent également très proches pour toutes les valeurs de m . L'écart relatif maximal est observé pour $m = 5000$ et reste inférieur à 0.7 %. Pour $m = 10000$ et $m = 20000$, les différences sont très faibles.

Ces résultats confirment les observations obtenues lors de la variation de k . La construction distribuée du coresets fournit, dans les configurations étudiées, une approximation dont la qualité reste proche de celle de l'implémentation séquentielle.

5.2.4 Limites de l'évaluation

Les résultats précédents doivent être interprétés en tenant compte de certaines limites du protocole expérimental.

Chaque configuration a été exécutée une seule fois avec une graine fixée à 2026. Ce choix permet de reproduire les expériences, mais ne permet pas de mesurer la variabilité associée aux étapes aléatoires de StreamKM++ et de k-means++.

Une évaluation plus complète pourrait répéter chaque configuration avec plusieurs graines et présenter notamment la moyenne et l'écart-type des SSE obtenues.

La seconde limite concerne la métrique utilisée. La SSE mesure la compacité des clusters, mais ne permet pas à elle seule d'évaluer de manière absolue la pertinence du partitionnement. Dans ce projet, elle est principalement utilisée comme métrique commune afin de comparer les différentes implémentations dans des conditions identiques.

5.3 Optimisation du noyau local

En complément de l'étude de la qualité du clustering, une expérimentation a été menée sur la représentation mémoire des points. Elle concerne les composants locaux de l'algorithme et est indépendante de l'exécution distribuée avec Spark.

L'objectif est de déterminer si une organisation différente des coordonnées permet d'améliorer les performances des opérations fréquemment exécutées lors du calcul des distances, du clustering et de la construction des coresets.

5.3.1 Représentations étudiées

L'implémentation initiale utilise une représentation de type *Array of Structures* (AoS). Les points sont stockés dans un `Array[Point]` et chaque objet `Point` contient son propre `Array[Double]`.

Une seconde représentation de type *Structure of Arrays* (SoA) a été développée dans `streamkm.core.Points`. Les coordonnées sont stockées de manière contiguë dans un `DoubleBuffer` reposant sur un `ByteBuffer.allocateDirect`.

L'hypothèse étudiée est qu'une organisation plus contiguë des coordonnées peut améliorer la localité mémoire et réduire le coût des accès lors des opérations intensives.

5.3.2 Protocole JMH

Les deux représentations ont été comparées à l'aide de JMH 1.37.

Les benchmarks ont été exécutés avec cinq itérations de préchauffage, trente itérations de mesure et un fork, dans le même environnement Java pour les deux représentations.

Six opérations ont été évaluées :

- `sqdist`;
- `nearest`;
- `BucketManager.insertAll`;
- `CoresetTree.build`;
- `KMeans.lloyd`;

— le pipeline local complet.

Ces benchmarks portent uniquement sur le noyau local de l’algorithme. Ils ne permettent donc pas de tirer directement des conclusions sur les performances d’un pipeline Spark complet.

5.3.3 Résultats

Contrairement à l’hypothèse initiale, la représentation SoA reposant sur un buffer hors tas s’est révélée plus lente que la représentation AoS pour l’ensemble des opérations évaluées.

Benchmark	Nature	Écart SoA par rapport à AoS
<code>sqdist</code>	calcul de distance	+46 à +77 %
<code>nearest</code>	recherche du centre	+45 à +51 %
<code>insertAll</code>	construction du résumé	+32 à +50 %
<code>CoresetTree.build</code>	construction du coreset	+36 à +59 %
<code>lloyd</code>	clustering	+20 à +30 %
<code>fullPipeline</code>	pipeline complet	+15 à +21 %

TABLE 5.3 – Écart relatif du temps moyen entre la représentation SoA hors tas et la représentation AoS.

Les écarts les plus importants sont observés pour les opérations directement dépendantes des accès aux coordonnées, notamment `sqdist` et `nearest`. Ils sont moins élevés lorsque le benchmark comprend davantage de traitements qui ne dépendent pas directement de la représentation mémoire, comme dans l’algorithme de Lloyd ou le pipeline complet.

Dans l’implémentation étudiée, le bénéfice attendu d’un stockage contigu des coordonnées ne compense donc pas le coût des accès réalisés à travers `DoubleBuffer`.

Ce résultat ne signifie pas qu’une représentation SoA est systématiquement moins performante. Il montre uniquement que la variante fondée sur `ByteBuffer.allocateDirect` et `DoubleBuffer` n’apporte pas d’amélioration dans le contexte testé.

Une représentation SoA utilisant un tableau `Array[Double]` plat conservé sur le tas pourrait permettre de distinguer plus précisément l’effet de l’organisation des données de celui du stockage hors tas. Cette variante n’a pas été étudiée dans le cadre du projet.

La représentation AoS initiale a donc été conservée.

5.4 Synthèse

Les expériences réalisées sur Covertypes montrent que les versions séquentielle et distribuée de StreamKM++ produisent des résultats très proches. Pour les valeurs de k étudiées, l’écart relatif entre les deux implémentations reste inférieur à 0.7 %. Lors de la variation de la taille du coreset, les différences observées restent également faibles.

La comparaison avec Spark MLlib montre que StreamKM++ obtient des SSE proches de celles d’un K-Means appliqué directement aux données distribuées. Dans les configurations testées, l’utilisation d’un coreset permet donc de réduire les données utilisées pour déterminer les centres sans entraîner de dégradation importante de la qualité du clustering.

L’étude de la taille du coreset montre également que, pour l’exécution réalisée, l’augmentation de m s’accompagne d’une diminution de la SSE. L’amélioration est surtout visible entre les plus petites tailles de coreset, puis devient plus limitée.

Enfin, les benchmarks réalisés sur la représentation mémoire montrent que la variante SoA fondée sur un `DoubleBuffer` hors tas est moins performante que la représentation AoS initiale dans les configurations étudiées.

Dans l’ensemble, les résultats obtenus montrent que l’implémentation distribuée de StreamKM++ conserve une qualité de clustering proche de celle de l’implémentation séquentielle sur Covertypes.

Les conclusions restent néanmoins limitées par l'utilisation d'une seule exécution par configuration et pourraient être consolidées par des répétitions avec plusieurs graines.

Chapitre 6

Discussion

Ce chapitre met en perspective les choix réalisés lors du portage de StreamKM++ vers Apache Spark. L’objectif est de revenir sur les principaux apports de l’implémentation et d’en présenter les limites à partir des résultats expérimentaux obtenus.

6.1 Apports de l’implémentation

Une construction du coreset adaptée au traitement de flux. L’un des principaux intérêts de StreamKM++ réside dans sa capacité à construire progressivement un résumé des observations sans conserver l’ensemble du flux en mémoire. Les données sont intégrées au mécanisme de *merge-and-reduce* présenté au chapitre 2, qui maintient un ensemble de résumés pondérés de taille contrôlée.

La quantité de données conservée dépend principalement de la taille m du coreset et du nombre de niveaux utilisés par la structure de buckets, plutôt que directement du nombre total d’observations déjà traitées. Ce fonctionnement permet d’envisager le traitement de flux dont la taille totale n’est pas connue à l’avance.

Il convient toutefois de distinguer la construction du coreset du protocole utilisé pour évaluer la qualité du clustering. Dans les expériences réalisées, la SSE est calculée sur l’ensemble des observations originales après la détermination des centres. Ce parcours supplémentaire appartient à l’évaluation du modèle et non à la construction du coreset elle-même.

Une séparation entre logique algorithmique et distribution. L’architecture du projet sépare les composants algorithmiques des mécanismes propres à Spark. Les packages `streamkm.core` et `streamkm.coreset` regroupent les opérations de clustering et de construction des coresets, tandis que `streamkm.spark` prend en charge leur exécution distribuée.

Cette organisation a permis de valider les principales propriétés de l’algorithme indépendamment de Spark avant son intégration dans le pipeline distribué. Les mêmes composants, notamment `BucketManager`, `CoresetTree` et `KMeans`, sont ainsi utilisés par les versions séquentielle et distribuée.

Les résultats expérimentaux présentés au chapitre 5 montrent que les deux implémentations produisent des SSE très proches dans les configurations étudiées. Cette proximité constitue un indicateur de la cohérence de l’adaptation réalisée.

Une construction distribuée par partition. Le modèle de partitionnement de Spark s’intègre naturellement à la construction des coresets. Chaque partition construit indépendamment un résumé local à partir des observations qui lui sont attribuées. Les différents résumés sont ensuite fusionnés afin de produire le coreset global.

Cette organisation permet d'effectuer la construction locale directement sur les partitions contenant les données. Seuls les résumés intermédiaires doivent ensuite être combinés lors de la phase de réduction.

Les résultats obtenus sur Coverttype montrent que cette décomposition ne conduit pas, dans les configurations testées, à une dégradation notable de la qualité par rapport à l'implémentation séquentielle.

Une fusion adaptée au modèle Spark. La méthode `treeReduce` permet de combiner progressivement les `BucketManager` produits par les différentes partitions. La fusion est organisée sous forme arborescente plutôt que par une récupération immédiate de l'ensemble des résumés sur le driver.

Ce choix correspond au modèle de réduction fourni par Spark et permet de réutiliser directement la méthode de fusion définie dans le mécanisme *merge-and-reduce*.

Cette organisation constitue un choix architectural du portage. Son impact précis sur les performances n'a toutefois pas été isolé expérimentalement dans le cadre du projet et ne peut donc pas être quantifié à partir des résultats présentés.

Une évaluation distribuée de la qualité. L'évaluation du clustering est également réalisée de manière distribuée. Les centres obtenus sont transmis aux exécuteurs à l'aide d'une variable `broadcast`. Chaque partition calcule ensuite la contribution de ses observations à la SSE, avant l'agrégation des résultats partiels.

Cette séparation entre la construction des centres à partir du coreset et leur évaluation sur les données originales permet de comparer les différentes méthodes sur une métrique commune.

Une adaptation aux mécanismes propres à Spark. Le portage de StreamKM++ ne consiste pas uniquement à remplacer les opérateurs Flink par des opérations Spark équivalentes. L'orchestration du traitement est adaptée au modèle d'exécution du framework.

Structured Streaming fournit notamment les micro-batches utilisés comme unités de traitement successives, tandis que les mécanismes `broadcast` et `treeReduce` sont utilisés respectivement pour diffuser les centres et combiner les résumés intermédiaires.

L'implémentation conserve ainsi la logique algorithmique de StreamKM++ tout en adaptant son exécution aux abstractions fournies par Spark.

6.2 Limites de l'implémentation

Le clustering final reste exécuté sur le driver. Après la construction du coreset global, `KMeans.fit` est exécuté sur le driver. Cette étape n'est donc pas distribuée dans l'implémentation actuelle.

Ce choix reste cohérent avec le principe de StreamKM++, puisque le clustering final est réalisé sur un résumé dont la taille est nettement inférieure à celle du jeu de données original. Son coût peut toutefois augmenter lorsque la taille m du coreset, le nombre de clusters k ou le nombre de redémarrages de k-means++ augmentent.

L'ajout de ressources Spark n'agit donc pas directement sur cette partie du traitement.

La version Structured Streaming maintient un état global sur le driver. Dans l'adaptation avec Structured Streaming, chaque micro-batch produit un coreset distribué qui est ensuite intégré dans un `BucketManager` global conservé sur le driver.

Cette approche simplifie la conservation d'un résumé entre les micro-batches, mais introduit également un point central dans l'architecture.

La version streaming constitue ainsi une adaptation du principe général de StreamKM++ au traitement par micro-batches. Elle ne reproduit pas exactement l'ordre d'insertion qui serait obtenu en alimentant directement une structure unique avec l'ensemble des observations brutes.

Latence liée au traitement par micro-batches. Structured Streaming est utilisé en mode *micro-batch*. Les observations sont donc prises en compte lors de l'exécution du lot auquel elles appartiennent et non nécessairement au moment exact de leur arrivée.

La latence dépend ainsi de la fréquence de déclenchement des micro-batches et du temps nécessaire à leur traitement. Cette caractéristique distingue l'implémentation Spark d'un traitement événement par événement.

Elle devient principalement importante pour les applications nécessitant une réaction à très faible latence. Elle est moins contraignante lorsque l'objectif consiste à mettre à jour périodiquement un résumé du flux.

État interne non directement exposé. L'implémentation retenue conserve le `BucketManager` global à l'intérieur du traitement Structured Streaming. Cet état n'est pas directement exposé pour être interrogé depuis l'extérieur du job.

Les modèles produits peuvent être transmis à une fonction ou à un mécanisme de sortie, mais l'état interne de la structure de coreset reste encapsulé dans le traitement.

Cette différence doit être prise en compte lors de la comparaison avec l'implémentation Flink de référence, qui s'appuie sur ses propres mécanismes de gestion d'état.

Sensibilité au partitionnement et à l'aléatoire. La construction distribuée dépend du partitionnement des données. Chaque partition traite un sous-ensemble différent des observations et construit son propre coreset avant la fusion globale.

Pour un même partitionnement et les mêmes graines, l'exécution peut être reproduite. Modifier le nombre de partitions modifie cependant la décomposition des données et peut donc conduire à des résumés intermédiaires et à un coreset final différents.

Cette sensibilité est également liée aux étapes aléatoires présentes dans la construction du coreset et dans l'initialisation k-means++. Elle justifie le contrôle du partitionnement et des graines dans les expériences.

Variabilité liée à la taille du coreset. Les expériences réalisées montrent une diminution de la SSE lorsque la taille m du coreset augmente dans les configurations testées. L'amélioration est particulièrement visible entre les plus petites tailles, puis devient plus limitée lorsque le coreset augmente.

Cette tendance reste toutefois observée sur une seule exécution par configuration avec une graine fixe. Elle ne permet donc pas de conclure à une diminution systématique de la SSE pour toute exécution.

Une étude plus complète nécessiterait plusieurs répétitions avec différentes graines afin de mesurer la variabilité des résultats et de caractériser plus précisément l'influence de m .

Absence de standardisation des variables de Covertime. Les expériences utilisent les 54 variables de Covertime sans standardisation préalable afin de conserver un protocole identique pour toutes les méthodes comparées.

Les attributs du jeu de données n'ont cependant pas tous la même échelle. Certaines variables prennent des valeurs relativement élevées, tandis que d'autres correspondent à des indicateurs binaires. Dans le calcul d'une distance euclidienne au carré, les variables de plus grande amplitude peuvent donc avoir une influence plus importante.

Une standardisation modifierait la géométrie du problème de clustering et pourrait conduire à des résultats différents. Son influence n'a pas été étudiée dans le cadre du projet.

Absence de mécanisme d’oubli. Le mécanisme *merge-and-reduce* de StreamKM++ construit un résumé de l’ensemble des observations traitées depuis le début du flux. Les données anciennes continuent donc à contribuer au coresé lorsque de nouvelles observations sont ajoutées.

Ce fonctionnement est adapté lorsque la distribution des données reste relativement stable. Il peut en revanche devenir moins adapté lorsque cette distribution évolue au cours du temps.

L’utilisation d’une fenêtre glissante ou d’un mécanisme donnant davantage de poids aux observations récentes constituerait une extension possible, mais n’entre pas dans le périmètre de l’implémentation réalisée.

6.3 Bilan

Le portage réalisé montre que les principaux composants de StreamKM++ peuvent être réutilisés dans une architecture Spark sans modification profonde de leur logique algorithmique. La construction de coresets locaux sur les partitions et leur fusion permettent d’obtenir une qualité de clustering très proche de celle de l’implémentation séquentielle dans les configurations testées.

La solution conserve néanmoins certains traitements centralisés, notamment le clustering final et l’état global utilisé entre les micro-batches. Les résultats obtenus doivent également être interprétés en tenant compte du caractère aléatoire de l’algorithme et du protocole expérimental limité à une exécution par configuration.

Dans l’ensemble, l’implémentation permet d’adapter StreamKM++ au modèle de traitement de Spark tout en conservant la qualité du clustering observée avec la version séquentielle. Les limites identifiées ouvrent plusieurs pistes d’amélioration, présentées dans le chapitre suivant.

Chapitre 7

Conclusion et perspectives

Ce rapport a présenté le portage de StreamKM++ depuis l’implémentation Flink proposée par Bitsakis (2018) [1] vers Apache Spark. Les principaux composants algorithmiques, notamment l’algorithme de Lloyd pondéré, l’initialisation k-means++, la construction du coreset et la stratégie *merge-and-reduce*, ont d’abord été implémentés et validés indépendamment de Spark. Ils ont ensuite été intégrés à une couche distribuée reposant sur les RDD, `treeReduce` et Structured Streaming.

L’implémentation retenue sépare la logique algorithmique des mécanismes propres au framework distribué. Chaque partition Spark peut ainsi construire un résumé local des données qui lui sont attribuées, avant que les différents coresets soient fusionnés pour produire un résumé global. Structured Streaming permet d’appliquer le même principe à des micro-batches successifs tout en maintenant un coreset représentant les observations déjà traitées.

L’évaluation réalisée sur le jeu de données Coverttype montre que la version distribuée conserve une qualité de clustering très proche de celle de l’implémentation séquentielle. Pour les valeurs de k étudiées, l’écart relatif entre leurs SSE reste inférieur à 0.7%. La variation de la taille du coreset conduit également à des résultats proches entre les deux versions, avec un écart maximal d’environ 0.63%. Les coûts obtenus restent par ailleurs proches de ceux produits par Spark MLlib, malgré l’utilisation d’un résumé des données dans StreamKM++.

Ces résultats montrent que la distribution de la construction du coreset peut être réalisée avec Spark sans dégradation notable de la qualité du clustering dans les configurations étudiées. Ils doivent néanmoins être interprétés dans les limites du protocole expérimental, notamment l’utilisation d’une seule exécution par configuration et d’une graine fixe.

L’expérimentation complémentaire sur la représentation mémoire des points a également montré que la variante SoA fondée sur un `DoubleBuffer` hors tas n’apportait pas d’amélioration par rapport à la représentation AoS initiale. Celle-ci a donc été conservée pour l’implémentation finale.

7.1 Perspectives

Plusieurs prolongements peuvent être envisagés à partir de ce travail.

Une première perspective consiste à approfondir l’évaluation expérimentale en répétant les différentes configurations avec plusieurs graines. Cette approche permettrait de mesurer la variabilité liée aux étapes aléatoires de la construction du coreset et de k-means++, et de consolider les comparaisons entre les versions séquentielle et distribuée.

L’influence de la taille du coreset pourrait également être étudiée plus finement. Les expériences réalisées montrent une amélioration de la SSE lorsque m augmente, mais avec des gains qui deviennent progressivement plus faibles. Des essais supplémentaires permettraient de mieux caractériser le compromis entre la taille du résumé, la qualité du clustering et le coût des traitements.

Une autre perspective concerne la représentation mémoire. Les benchmarks réalisés montrent que l'utilisation d'un `DoubleBuffer` hors tas est moins performante dans le contexte étudié. Une représentation SoA reposant sur un tableau `Array[Double]` plat conservé sur le tas pourrait être évaluée afin de distinguer l'effet de la disposition des données de celui du stockage hors tas.

Enfin, l'adaptation à des flux dont la distribution évolue au cours du temps constitue une extension possible. Le mécanisme actuel conserve l'influence de l'ensemble des observations passées. L'introduction d'une fenêtre glissante ou d'un mécanisme de décroissance des poids permettrait d'accorder davantage d'importance aux données récentes dans les contextes soumis à une évolution de la distribution.

Dans son périmètre actuel, ce projet fournit ainsi une implémentation de `StreamKM++` adaptée à Apache Spark et validée expérimentalement sur un jeu de données réel. Il montre qu'une construction distribuée du coreset peut préserver une qualité de clustering proche de celle obtenue avec l'implémentation séquentielle, tout en conservant une architecture compatible avec le modèle de traitement de Spark.

Annexe A

Code source complet

Cette annexe inclut, via `\lstinputlisting`, le contenu réel des principaux fichiers du dépôt `StreamKMpp` utilisés dans l'implémentation.

A.0.1 Configuration du projet

```
1 // Scala 2.12 est imposé par Spark 3.5 (E4). On s'y tient dès maintenant pour éviter
2 // une migration au moment d'ajouter la couche Spark.
3 ThisBuild / scalaVersion := "2.12.18"
4 ThisBuild / organization := "streamkm"
5 ThisBuild / version      := "0.1.0-SNAPSHOT"
6
7 // Spark 3.5.x : dernière branche 3.5 supportant nativement Java 17 (E4).
8 // Scope "provided" : disponible en compile/test, absent du jar assemblé - c'est
9 // spark-submit qui fournit ces classes au runtime en production.
10 val sparkVersion = "3.5.3"
11
12 lazy val root = (project in file("."))
13   .settings(
14     name := "streamkm-spark",
15     libraryDependencies ++= Seq(
16       "org.scalatest" %% "scalatest" % "3.2.18" % Test,
17       "org.apache.spark" %% "spark-core" % sparkVersion % Provided,
18       "org.apache.spark" %% "spark-sql" % sparkVersion % Provided,
19       // Ajouté pour streamkm.experiments.QualityMetrics : sert de référence externe
20       // (org.apache.spark.ml.clustering.KMeans, entraîné sur le dataset complet, pas
21       // sur un coreset) dans l'expérience de comparaison à MLlib.
22       "org.apache.spark" %% "spark-mllib" % sparkVersion % Provided
23     ),
24     scalacOptions ++= Seq(
25       "-deprecation",
26       "-feature",
27       "-unchecked",
28       "-Xlint"
29     ),
30     // -ea active les 'assert' d'invariants du BucketManager pendant les tests.
31     // En production (E4) on laisse -ea désactivé : les assertions coûtent cher
32     // dans les boucles chaudes.
33     //
34     // Les --add-opens sont indispensables pour faire tourner Spark sous Java 17 :
35     // Spark/Hadoop utilisent de la réflexion sur des modules internes du JDK
36     // (java.lang, java.nio, sun.nio.ch...) que le module system de Java 17 bloque
37     // par défaut (InaccessibleObjectException sans ce flag). Ce n'est PAS une
38     // fantaisie de config : sans ça, 'sbt test' plante dès la création du
39     // SparkContext. Spark lui-même ajoute ces mêmes flags dans ses scripts de
40     // lancement (spark-submit) à partir de la 3.3.
41     Test / fork := true,
42     Test / javaOptions ++= Seq(
43       "-Xmx2g",
44       "-ea",
45       "--add-opens=java.base/java.lang=ALL-UNNAMED",
46       "--add-opens=java.base/java.lang.invoke=ALL-UNNAMED",
47       "--add-opens=java.base/java.lang.reflect=ALL-UNNAMED",
48       "--add-opens=java.base/java.io=ALL-UNNAMED",
```

```

49     "--add-opens=java.base/java.net=ALL-UNNAMED",
50     "--add-opens=java.base/java.nio=ALL-UNNAMED",
51     "--add-opens=java.base/java.util=ALL-UNNAMED",
52     "--add-opens=java.base/java.util.concurrent=ALL-UNNAMED",
53     "--add-opens=java.base/java.util.concurrent.atomic=ALL-UNNAMED",
54     "--add-opens=java.base/sun.nio.ch=ALL-UNNAMED",
55     "--add-opens=java.base/sun.nio.cs=ALL-UNNAMED",
56     "--add-opens=java.base/sun.security.action=ALL-UNNAMED",
57     "--add-opens=java.base/sun.util.calendar=ALL-UNNAMED"
58 ),
59
60 // 'run'/'runMain' : mêmes flags JVM que Test (Spark sous Java 17), avec plus de
61 // mémoire (le harnais d'expériences streamkm.experiments charge Covertypes en
62 // entier, ~580k points, en mémoire locale pour la référence séquentielle).
63 run / fork := true,
64 run / javaOptions := (Test / javaOptions).value.map {
65     case "-Xmx2g" => "-Xmx4g"
66     case other    => other
67 },
68
69 // Limitation sbt connue (cf. CLAUDE.md, note sous "Commandes") : par défaut,
70 // 'Compile / run' ET 'Compile / runMain' n'exposent PAS les dépendances
71 // "Provided" (Spark) sur leur classpath runtime - seul 'Test / *' les voit.
72 // 'run' et 'runMain' sont deux tâches sbt DISTINCTES (découvert le
73 // 2026-08-22 : corriger seulement 'run' ne suffit pas pour
74 // 'sbt runMain ...') ; les deux sont donc réalignées sur
75 // 'Compile / fullClasspath' (qui inclut Provided), pour que 'sbt run' ET
76 // 'sbt runMain streamkm.experiments.QualityMetrics' fonctionnent directement
77 // sans devoir passer par 'Test/runMain'.
78 Compile / run := Defaults.runTask(
79     Compile / fullClasspath,
80     Compile / run / mainClass,
81     Compile / run / runner
82 ).evaluated,
83 Compile / runMain := Defaults.runMainTask(
84     Compile / fullClasspath,
85     Compile / run / runner
86 ).evaluated
87 )

```

Listing A.1 – build.sbt

A.0.2 Couche core : structures de données et k-means

```

1 package streamkm.core
2
3 /**
4  * Un point pondéré de l'espace euclidien de dimension  $d$ .
5  *
6  * 'weight' vaut 1.0 pour un point brut du flux, et la masse totale du sous-
7  * ensemble
8  * représenté pour un point issu d'un coreset (thèse §2.3.3 : « we denote the
9  * weight of
10  * each  $q_i$  as the total number of points in each subset  $P$  »).
11  *
12  * ATTENTION : 'coords' n'est pas copié défensivement, ni à la construction ni à
13  * la
14  * lecture. C'est un choix délibéré - copier à chaque accès tuerait les
15  * performances dans
16  * les boucles de 'Distance' et de 'CoresetTree'. La contrepartie est un
17  * invariant à
18  * respecter partout : **un tableau de coordonnées n'est jamais muté après
19  * construction**.
20  *
21  * Toute mutation potentielle passe par '.clone()' explicite.
22  *
23  * Cette classe reste volontairement une classe « objet » (Array of Structures)
24  * en E1/E2.

```

```

17  * L'étape E6 introduira une représentation Structure of Arrays derrière la même
    API
18  * publique, et JMH mesurera l'écart.
19  */
20  final class Point(val coords: Array[Double], val weight: Double) extends
    Serializable {
21
22      def dim: Int = coords.length
23
24      /** Utile uniquement pour les messages d'erreur et le débogage. */
25      override def toString: String =
26          coords.mkString("Point([", ", ", s"]", w=$weight)")
27  }
28
29  object Point {
30
31      /** Point brut du flux : poids 1. */
32      def apply(coords: Array[Double]): Point = new Point(coords, 1.0)
33
34      def apply(coords: Array[Double], weight: Double): Point = new Point(coords,
          weight)
35
36      /** Confort pour les tests : 'Point(1.0, 2.0)'. */
37      def of(coords: Double*): Point = new Point(coords.toArray, 1.0)
38
39      /**
40       * Parsing d'une ligne CSV/SSV en Point.
41       * Renvoie None si la ligne est malformée ou n'a pas la dimension attendue - c
42       * 'est
43       * l'équivalent du 'FlatMapToPoint' de la thèse (§4.1), qui utilise un flatMap
44       * précisément pour pouvoir rejeter les lignes invalides sans faire échouer le
45       * job.
46       */
47      def parse(line: String, dim: Int, sep: Char = ','): Option[Point] = {
48          val parts = line.trim.split(sep)
49          if (parts.length != dim) None
50          else {
51              val arr = new Array[Double](dim)
52              var i = 0
53              var ok = true
54              while (i < dim && ok) {
55                  try arr(i) = java.lang.Double.parseDouble(parts(i))
56                  catch { case _: NumberFormatException => ok = false }
57                  i += 1
58              }
59              if (ok) Some(new Point(arr, 1.0)) else None
60          }
61      }
62  }
63
64  object Distance {
65
66      /**
67       * Distance euclidienne AU CARRÉ.
68       *
69       * Toute la chaîne StreamKM++ (seeding  $D^2$ , coût du coresets tree, fonction
70       * objectif
71       * k-means) travaille sur des distances au carré. On ne calcule donc jamais de
72       * racine :
73       * elle serait à la fois inutile et coûteuse. C'est le point le plus chaud du
74       * programme.
75       */
76      def sqdist(a: Array[Double], b: Array[Double]): Double = {

```

```

72     var s = 0.0
73     var i = 0
74     val n = a.length
75     while (i < n) {
76         val d = a(i) - b(i)
77         s += d * d
78         i += 1
79     }
80     s
81 }
82
83 /**
84  * Indice du centre le plus proche et distance au carré correspondante, en un
85  * seul
86  * parcours.
87  * Le tuple alloue (et boxe le Int et le Double). C'est assumé en E1 : la
88  * lisibilité
89  * prime tant que la correction n'est pas établie. E6 remplacera cet appel par
90  * une
91  * boucle inline ou un encodage sans allocation, et JMH quantifiera le gain.
92  */
93 def nearest(p: Array[Double], centers: Array[Array[Double]]): (Int, Double) =
94 {
95     var best = 0
96     var bestD = Double.MaxValue
97     var c = 0
98     while (c < centers.length) {
99         val d = sqdist(p, centers(c))
100         if (d < bestD) { bestD = d; best = c }
101         c += 1
102     }
103     (best, bestD)
104 }

```

Listing A.2 – streamkm/core/Point.scala

```

1 package streamkm.core
2
3 import scala.util.Random
4
5 /** Résultat d'un clustering : les centres, la SSE pondérée atteinte, le nombre
6  * d'itérations. */
7 final case class KMeansModel(
8     centers:    Array[Array[Double]],
9     cost:      Double,
10    iterations: Int
11 ) extends Serializable {
12
13     def k: Int = centers.length
14
15     override def toString: String =
16         s"KMeansModel(k=$k, cost=$cost, iterations=$iterations)"
17 }
18
19 /**
20  * Échantillonnage pondéré, factorisé ici parce qu'il sert à trois endroits :
21  * le premier centre de k-means++, le premier représentant du coreset tree, et
22  * l'échantillonnage  $D^2$  à l'intérieur d'une feuille.
23  */
24 private[streamkm] object Sampling {
25
26     /**

```

```

26  * Tire un indice dans [0, n) avec une probabilité proportionnelle à 'w(i)'.
27  * Si la masse totale est nulle (ou négative par erreur d'arrondi), retombe
    sur
28  * un tirage uniforme plutôt que de lever une exception : ce cas se produit
29  * légitimement quand tous les points d'une feuille sont confondus.
30  */
31  def byWeight(n: Int, w: Int => Double, rng: Random): Int = {
32      var total = 0.0
33      var i      = 0
34      while (i < n) { total += w(i); i += 1 }
35
36      if (!(total > 0.0)) return rng.nextInt(n)
37
38      val target = rng.nextDouble() * total
39      var acc     = 0.0
40      i = 0
41      while (i < n) {
42          acc += w(i)
43          if (acc >= target) return i
44          i += 1
45      }
46      // Atteignable uniquement par accumulation d'erreurs flottantes.
47      n - 1
48  }
49 }
50
51 object KMeans {
52
53     /**
54      * Procédure de seeding de k-means++ (thèse, algorithme 2.2, lignes 1-3), **
55      * version pondérée**.
56      *
57      * Différence assumée avec la thèse : la ligne 1 de l'algorithme 2.2 choisit
58      * le premier
59      * centre « uniformly at random ». On tire ici proportionnellement au poids.
60      * Sur des points
61      * bruts (tous de poids 1) les deux coïncident exactement ; sur un coresé (
62      * poids très
63      * hétérogènes) le tirage uniforme biaiserait l'initialisation vers les ré
64      * gions peu denses.
65      * Or dans StreamKM++, k-means++ est TOUJOURS appliqué à un coresé (§2.3.4, é
66      * tape 2) -
67      * la version pondérée est donc la bonne. À mentionner dans le rapport.
68      *
69      * Coût :  $O(n \cdot k \cdot d)$ .
70      */
71     def seedPlusPlus(points: Array[Point], k: Int, rng: Random): Array[Array[
72         Double]] = {
73         require(points.nonEmpty, "seedPlusPlus: ensemble de points vide")
74         require(k > 0, s"seedPlusPlus: k doit être > 0 (reçu $k)")
75
76         val n      = points.length
77         val kEff    = math.min(k, n)
78         val centers = new Array[Array[Double]](kEff)
79
80         centers(0) = points(Sampling.byWeight(n, i => points(i).weight, rng)).coords
81             .clone()
82
83         // closest(i) = distance au carré de points(i) au plus proche centre déjà
84         // choisi.
85         // Mis à jour incrémentalement : c'est ce qui donne  $O(n \cdot k \cdot d)$  et non  $O(n \cdot k^2 \cdot d)$ .
86         val closest = new Array[Double](n)

```

```

78     var i          = 0
79     while (i < n) { closest(i) = Distance.sqdist(points(i).coords, centers(0));
      i += 1 }

80
81     var c = 1
82     while (c < kEff) {
83         // Tirage  $D^2$  pondéré :  $P(i) \propto w_i \cdot \text{closest}(i)$ 
84         val idx = Sampling.byWeight(n, j => points(j).weight * closest(j), rng)
85         centers(c) = points(idx).coords.clone()
86
87         i = 0
88         while (i < n) {
89             val d = Distance.sqdist(points(i).coords, centers(c))
90             if (d < closest(i)) closest(i) = d
91             i += 1
92         }
93         c += 1
94     }
95     centers
96 }

97
98 /**
99  * Algorithme de Lloyd pondéré (thèse, algorithme 2.1).
100  *
101  * Le centre d'un cluster est la moyenne PONDÉRÉE de ses points :
102  *  $\mu_c = \sum_{\{p \in c\}} w_p \cdot p / \sum_{\{p \in c\}} w_p$ 
103  * Oublier les poids ici est le bug classique de StreamKM++ : le code tourne,
      ne plante
104  * jamais, et produit des centres silencieusement faux dès qu'on l'applique à
      un coreset.
105  *
106  * Clusters vides : on conserve le centre précédent. C'est le choix le plus
      simple et il
107  * ne dégrade pas le coût (un cluster vide contribue 0). Alternative classique
      - ré-amorcer
108  * sur le point le plus éloigné - écartée : elle rend le coût non monotone et
      casse
109  * l'assertion de convergence.
110  *
111  * Arrêt : décroissance relative du coût inférieure à 'tol', ou 'maxIter'
      atteint.
112  */
113 def lloyd(
114     points: Array[Point],
115     init:    Array[Array[Double]],
116     maxIter: Int    = 100,
117     tol:     Double = 1e-6
118 ): KMeansModel = {
119     require(points.nonEmpty, "lloyd: ensemble de points vide")
120     require(init.nonEmpty, "lloyd: aucun centre initial")
121
122     val k          = init.length
123     val d          = points(0).dim
124     val centers     = Array.tabulate(k)(i => init(i).clone())
125
126     val sums       = Array.ofDim[Double](k, d)
127     val masses     = new Array[Double](k)
128
129     var prevCost   = Double.MaxValue
130     var it         = 0
131     var done       = false
132
133     while (!done && it < maxIter) {

```

```

134     var c = 0
135     while (c < k) {
136         java.util.Arrays.fill(sums(c), 0.0)
137         masses(c) = 0.0
138         c += 1
139     }
140
141     // Étape d'affectation + accumulation des sommes pondérées, en un seul
142     // 'currCost' est la SSE des centres COURANTS (avant mise à jour) : c'est
143     // bien la
144     // quantité dont la décroissance prouve la convergence de Lloyd.
145     var currCost = 0.0
146     var i = 0
147     while (i < points.length) {
148         val p = points(i)
149         val (idx, dd) = Distance.nearest(p.coords, centers)
150         currCost += p.weight * dd
151         val s = sums(idx)
152         val w = p.weight
153         var j = 0
154         while (j < d) { s(j) += w * p.coords(j); j += 1 }
155         masses(idx) += w
156         i += 1
157     }
158
159     // Étape de mise à jour des centres.
160     c = 0
161     while (c < k) {
162         if (masses(c) > 0.0) {
163             val s = sums(c)
164             val m = masses(c)
165             val ctr = centers(c)
166             var j = 0
167             while (j < d) { ctr(j) = s(j) / m; j += 1 }
168         }
169         c += 1
170     }
171
172     it += 1
173     // Invariant de Lloyd : le coût ne remonte jamais.
174     assert(currCost <= prevCost + 1e-9 * math.abs(prevCost),
175            s"Lloyd: coût non monotone ($prevCost -> $currCost) - bug d'
176            affectation ou de pondération")
177
178     if (prevCost - currCost <= tol * math.max(math.abs(prevCost), 1e-300))
179         done = true
180     prevCost = currCost
181 }
182
183 // Recalcul final : 'prevCost' correspond aux centres d'AVANT la dernière
184 // mise à jour.
185 KMeansModel(centers, cost(points, centers), it)
186 }
187
188 /**
189  * k-means++ complet : 'nRestarts' exécutions indépendantes (seeding + Lloyd),
190  * on garde
191  * la moins coûteuse. La thèse et le papier original utilisent 5 (§2.3.4, é
192  * tape 2), mais
193  * le §5.3.1 montre qu'une seule application suffit en pratique - c'est un
194  * paramètre
195  * d'expérience (E3), pas une constante.

```

```

189  */
190  def fit(
191    points:    Array[Point],
192    k:          Int,
193    nRestarts: Int    = 5,
194    maxIter:   Int    = 100,
195    tol:       Double = 1e-6,
196    seed:      Long   = 42L
197  ): KMeansModel = {
198    require(nRestarts > 0, s"fit: nRestarts doit être > 0 (reçu $nRestarts)")
199    val rng = new Random(seed)
200    var best: KMeansModel = null
201    var r = 0
202    while (r < nRestarts) {
203      val model = lloyd(points, seedPlusPlus(points, k, rng), maxIter, tol)
204      if (best == null || model.cost < best.cost) best = model
205      r += 1
206    }
207    best
208  }
209
210  /**
211   * Fonction objectif : SSE pondérée,  $\sum_p w_p * \min_c \|p - c\|^2$ .
212   * C'est la mesure de qualité unique de tout le projet (thèse §4.2) : tous les
213   * chapitres expérimentaux comparent des valeurs produites par cette fonction.
214   */
215  def cost(points: Array[Point], centers: Array[Array[Double]]): Double = {
216    var s = 0.0
217    var i = 0
218    while (i < points.length) {
219      val p = points(i)
220      s += p.weight * Distance.nearest(p.coords, centers)._2
221      i += 1
222    }
223    s
224  }
225 }

```

Listing A.3 – streamkm/core/KMeans.scala

A.0.3 Couche coreset : coreset tree et merge-and-reduce

```

1  package streamkm.coreset
2
3  import scala.collection.mutable.ArrayBuffer
4  import scala.util.Random
5
6  import streamkm.core.{Distance, Point, Sampling}
7
8  /**
9   * Nœud de l'arbre de coreset.
10  *
11  * NOTE DE NOMMAGE - importante, à reprendre dans le rapport.
12  * La thèse appelle 'weight(v)' l'attribut servant à la descente dans l'arbre,
13  *   et le définit
14  * pour une feuille comme 'cost(P_v, q_v)', c'est-à-dire une SOMME DE DISTANCES
15  *   AU CARRÉ.
16  * Mais elle appelle aussi « weight » le poids des points du coreset produit,
17  *   qui vaut le
18  * NOMBRE (la masse) de points représentés. Ce sont deux quantités totalement
19  *   différentes
20  * portant le même nom. On les sépare explicitement :

```

```

17 *   - 'cost' :  $\sum_{p \in P_v} w_p * ||p - q_v||^2$ 
18 *               sert a la descente
19 *
20 *   - 'mass' :  $\sum_{p \in P_v} w_p$ 
21 *               sert au poids du point de coreset en sortie
22 *
23 * @param points feuille : les points du cluster. Nœud interne : null.
24 * @param repr point représentatif  $q_v$  (stocké aussi sur les nœuds internes,
25 *   il devient
26 *               le représentant du fils gauche lors d'une scission).
27 */
28 private[coreset] final class Node(
29   var points: Array[Point],
30   var repr:   Array[Double],
31   var cost:   Double,
32   var mass:   Double,
33   var left:   Node,
34   var right:  Node
35 ) {
36   def isLeaf: Boolean = left == null
37 }
38 /**
39  * Construction du coreset par arbre binaire de clustering divisif hiérarchique
40  * (thèse §2.3.3 « Coreset Tree » et algorithme 2.4).
41  *
42  * Complexité :  $O(m \cdot n)$  dans le pire cas, mais  $O(n \cdot \log m)$  en pratique, contre
43  *  $O(n \cdot m)$ 
44  * pour l'AdaptiveCoreset de l'algorithme 2.3 - c'est toute la raison d'être de
45  * cette
46  * structure.
47  */
48 object CoresetTree {
49   /**
50    * Construit un coreset d'au plus 'm' points pondérés à partir de 'points'.
51    *
52    * GARANTIE STRUCTURELLE : la somme des poids de sortie est exactement égale à
53    * la somme
54    * des poids d'entrée. C'est l'invariant que testent en priorité les tests de
55    * E2 - il
56    * attrape la grande majorité des erreurs d'implémentation de cette structure.
57    *
58    * Si 'points.length <= m', l'ensemble est déjà son propre coreset : on le
59    * renvoie tel quel.
60    */
61   def build(points: Array[Point], m: Int, rng: Random): Array[Point] = {
62     require(m > 0, s"build: m doit être > 0 (reçu $m)")
63     if (points.length <= m) return points.clone()
64
65     // --- Racine : premier représentant tiré proportionnellement au poids ---
66     // (Thèse algo 2.4 ligne 1 : « uniformly at random ». Même remarque que pour
67     // KMeans.seedPlusPlus : sur un ensemble pondéré, le tirage proportionnel à
68     // la masse
69     // est la généralisation correcte, et les deux coïncident sur des points de
70     // poids 1.)
71     val i0 = Sampling.byWeight(points.length, i => points(i).weight, rng)
72     val root = new Node(points, points(i0).coords.clone(), 0.0, 0.0, null, null)
73     recomputeLeaf(root)
74
75     var leaves = 1
76     val path = new ArrayBuffer[Node](32)

```

```

72 // Condition d'arrêt supplémentaire par rapport à l'algorithme 2.4 : 'root.
    cost > 0'.
73 // Si le coût global est nul, tous les points sont confondus avec leur repré
    sentant :
74 // aucune scission ne peut plus rien améliorer, et l'échantillonnage  $D^2$ 
    serait dégénéré.
75 // Ce cas arrive réellement sur des données discrètes très redondantes.
76 while (leaves < m && root.cost > 0.0) {
77     path.clear()
78     val leaf = descend(root, rng, path)
79     val q     = sampleD2(leaf, rng)
80
81     if (q == null) return collectLeaves(root) // feuille de coût nul : sécurit
        é
82     if (!split(leaf, q)) return collectLeaves(root) // scission dégénérée : sé
        curité
83
84     // Remontée de la mise à jour jusqu'à la racine (algo 2.4 ligne 9).
85     // 'leaf' est le dernier élément du chemin et a déjà été mis à jour par '
        split'.
86     var i = path.length - 2
87     while (i >= 0) { recomputeInner(path(i)); i -= 1 }
88
89     leaves += 1
90 }
91
92 collectLeaves(root)
93 }
94
95 // -----
96 // Étapes internes (numérotées comme au §2.3.3)
97 // -----
98
99 /**
100  * Étape 1 : choisir une feuille avec une probabilité proportionnelle à son co
    ût.
101  *
102  *
103  * Le §2.3.3 étape 1 dit « with a probability proportional to cost( $P_\ell$ ,  $q_\ell$ ) »
    , et détaille
104  * une descente où un fils est choisi avec probabilité  $\text{cost}(\text{fils})/\text{cost}(\text{père})$ .
    La ligne 5 de
105  * l'algorithme 2.4 dit en revanche « according to their weights ». Les deux
    formulations
106  * s'accordent si l'on comprend «  $\text{weight}(v)$  » au sens de la définition donnée
    juste avant
107  * l'algorithme ( $\text{weight}$  d'une feuille =  $\text{cost}(P_v, q_v)$ ) - ce que confirme le
    papier original
108  * d'Ackermann et al. C'est donc bien le COÛT qui pilote la descente.
109  *
110  * Le raisonnement le confirme : une feuille déjà bien représentée (coût
    faible) ne doit
111  * plus être scindée ; l'effort de raffinement doit aller là où l'erreur est
    grande. Utiliser
112  * la masse à la place ferait dériver l'algorithme vers un simple é
    chantillonnage
113  * proportionnel à la densité, et lui ferait perdre la propriété de coresets.
114  *
115  * 'path' reçoit le chemin racine → feuille, pour la remontée.
116  */
117 private def descend(root: Node, rng: Random, path: ArrayBuffer[Node]): Node =
    {
118     var node = root

```

```

119     path += node
120     while (!node.isLeaf) {
121         val total = node.cost
122         node =
123             if (!(total > 0.0)) node.left // sécurité : coût nul, choix arbitraire
124             else if (rng.nextDouble() * total < node.left.cost) node.left
125             else node.right
126         path += node
127     }
128     node
129 }
130
131 /**
132  * Étape 2 : dans la feuille, tirer le prochain représentant avec une
133     probabilité
134  * proportionnelle à sa distance au carré au représentant courant (é
135     chantillonnage  $D^2$ ).
136  * Renvoie null si la feuille est de coût nul.
137  */
138 private def sampleD2(leaf: Node, rng: Random): Array[Double] = {
139     if (!(leaf.cost > 0.0)) return null
140     val pts = leaf.points
141     val idx = Sampling.byWeight(
142         pts.length,
143         i => pts(i).weight * Distance.sqdist(pts(i).coords, leaf.repr),
144         rng
145     )
146     val chosen = pts(idx).coords
147     // Si le tirage retombe sur le représentant lui-même (distance nulle), la
148     // scission
149     // produirait un fils vide. Ne peut arriver que par erreur d'arrondi ; on
150     // refuse.
151     if (Distance.sqdist(chosen, leaf.repr) <= 0.0) null else chosen.clone()
152 }
153
154 /**
155  * Étape 3 : scinder la feuille en deux fils selon la proximité à 'repr' et à
156     'q',
157  * puis mettre à jour les attributs du nœud devenu interne.
158  *
159  * Les deux fils sont nécessairement non vides : 'repr' est à distance 0 de
160     lui-même
161  * (donc à gauche) et 'q' est à distance 0 de lui-même et à distance > 0 de '
162     repr'
163  * (donc à droite). Le 'false' de retour est une sécurité contre une violation
164     de
165  * cette propriété par erreur numérique.
166  */
167 private def split(leaf: Node, q: Array[Double]): Boolean = {
168     val pts = leaf.points
169     val left = new ArrayBuffer[Point](pts.length)
170     val right = new ArrayBuffer[Point](pts.length)
171
172     var i = 0
173     while (i < pts.length) {
174         val p = pts(i)
175         if (Distance.sqdist(p.coords, leaf.repr) <= Distance.sqdist(p.coords, q))
176             left += p
177         else right += p
178         i += 1
179     }
180
181     if (left.isEmpty || right.isEmpty) return false

```

```

173
174     val l = new Node(left.toArray, leaf.repr, 0.0, 0.0, null, null)
175     val r = new Node(right.toArray, q, 0.0, 0.0, null, null)
176     recomputeLeaf(l)
177     recomputeLeaf(r)
178
179     leaf.points = null
180     leaf.left   = l
181     leaf.right  = r
182     recomputeInner(leaf)
183     true
184 }
185
186 private def recomputeLeaf(n: Node): Unit = {
187     var cost = 0.0
188     var mass = 0.0
189     val pts  = n.points
190     var i    = 0
191     while (i < pts.length) {
192         val p = pts(i)
193         cost += p.weight * Distance.sqdist(p.coords, n.repr)
194         mass += p.weight
195         i += 1
196     }
197     n.cost = cost
198     n.mass = mass
199 }
200
201 private def recomputeInner(n: Node): Unit = {
202     n.cost = n.left.cost + n.right.cost
203     n.mass = n.left.mass + n.right.mass
204 }
205
206 /**
207  * L'union des représentants des feuilles constitue le coreset. Le poids de
208  * chacun est
209  * la masse totale des points de sa feuille - d'où la conservation de la masse
210  * totale.
211  */
212 private def collectLeaves(root: Node): Array[Point] = {
213     val out  = new ArrayBuffer[Point]()
214     val stack = new ArrayBuffer[Node]()
215     stack += root
216     while (stack.nonEmpty) {
217         val n = stack.remove(stack.length - 1)
218         if (n.isLeaf) out += new Point(n.repr, n.mass)
219         else { stack += n.left; stack += n.right }
220     }
221     out.toArray
222 }

```

Listing A.4 – streamkm/coreset/CoresetTree.scala

```

1 package streamkm.coreset
2
3 import scala.collection.mutable.ArrayBuffer
4 import scala.util.Random
5
6 import streamkm.core.Point
7
8 /**
9  * Technique « merge-and-reduce » de StreamKM++ (thèse §2.3.4, algorithme 2.5).
10  *

```

```

11 * INVARIANTS (thèse §2.3.4, propriétés 1 et 2) :
12 *   - 'bucket(0)' est le tampon d'entrée : entre 0 et m points.
13 *   - 'bucket(i)' pour  $i \geq 1$  est soit VIDE, soit exactement de taille m.
14 *   - un 'bucket(i)' plein pour  $i \geq 1$  représente exactement  $2^{i-1} \cdot m$  points
    du flux.
15 *   - le nombre de buckets croît en  $O(\log(n/m))$  : on n'a pas besoin de connaî
    tre n a priori,
16 *     les niveaux sont créés à la demande.
17 *
18 * Ces invariants sont vérifiés par 'checkInvariants()', appelé sous 'assert' (
    donc actif
19 * seulement avec -ea, cf. build.sbt).
20 *
21 * Cette classe est l'état que la thèse maintient dans chaque subtask parallèle
    de
22 * 'FlatMapToPartialCoresets' (§4.1). Côté Spark, c'est l'accumulateur d'un '
    treeAggregate'
23 * (E4) : 'seqOp = insert', 'combOp = merge'.
24 *
25 * NON THREAD-SAFE - un 'BucketManager' appartient à une seule partition.
26 */
27 final class BucketManager(val m: Int, val seed: Long) extends Serializable {
28     require(m > 0, s"BucketManager: m doit être > 0 (reçu $m)")
29
30     private val rng = new Random(seed)
31     private val buckets: ArrayBuffer[ArrayBuffer[Point]] = ArrayBuffer(new
        ArrayBuffer[Point](m))
32     private var seen: Double = 0.0
33     private var collapsed: Boolean = false
34
35     /** Masse totale consommée depuis la création ( $\Sigma$  des poids insérés). */
36     def numPointsSeen: Double = seen
37
38     /** Nombre de niveaux actuellement alloués ( $\approx \log_2(n/m) + 2$ ). */
39     def numLevels: Int = buckets.length
40
41     /**
42      * Insère un point dans le flux (algorithme 2.5).
43      *
44      * Déroulé, à partir de la description en prose du §2.3.4 :
45      *   - le point va toujours dans B0 ;
46      *   - si B0 est plein et B1 vide, on déplace B0 dans B1 (aucun coreset n'est
        calculé,
47      *     B1 contient m points bruts et représente donc bien  $2^0 \cdot m$  points) ;
48      *   - si B1 est plein, on construit un coreset de  $B0 \cup B1$  ( $2m$  points  $\rightarrow m$ 
        points) et on
49      *     le range dans B2 ; si B2 est plein, on recommence avec B2, et ainsi de
        suite jusqu'à
50      *     trouver un niveau libre.
51      */
52     def insert(p: Point): Unit = {
53         if (collapsed)
54             throw new IllegalStateException(
55                 "BucketManager: insertion interdite après merge(). Un manager fusionné
                    est un " +
56                 "résultat de combOp, il n'est plus alimenté par le flux."
57             )
58
59         buckets(0) += p
60         seen += p.weight
61
62         if (buckets(0).length >= m) {
63             ensureLevel(1)

```

```

64     if (buckets(1).isEmpty) {
65         // B1 vide : simple déplacement, pas de réduction.
66         buckets(1) += buckets(0)
67         buckets(0).clear()
68     } else {
69         // B1 plein : cascade de fusions-réductions.
70         var union: Array[Point] = (buckets(0) ++ buckets(1)).toArray
71         buckets(0).clear()
72         buckets(1).clear()
73
74         var level = 2
75         var placed = false
76         while (!placed) {
77             val reduced = CoresetTree.build(union, m, rng)
78             ensureLevel(level)
79             if (buckets(level).isEmpty) {
80                 buckets(level) += reduced
81                 placed = true
82             } else {
83                 union = reduced ++ buckets(level)
84                 buckets(level).clear()
85                 level += 1
86             }
87         }
88     }
89     assert(checkInvariants(), "BucketManager: invariants violés après insert")
90 }
91 }
92
93 /** Insertion en lot. Renvoie 'this' pour pouvoir chaîner dans un 'seqOp'. */
94 def insertAll(it: Iterator[Point]): this.type = {
95     while (it.hasNext) insert(it.next())
96     this
97 }
98
99 def insertAll(pts: Array[Point]): this.type = insertAll(pts.iterator)
100
101 /**
102  * Union de tous les buckets non vides, SANS réduction.
103  *
104  * C'est le « final coreset » au sens du §2.3.4 : au plus  $m \cdot (\log_2(n/m) + 2)$ 
105  * points. C'est cet ensemble que la thèse passe à k-means++ à l'étape 2 de
106  * StreamKM++.
107  */
108 def union(): Array[Point] = {
109     val out = new ArrayBuffer[Point]()
110     var i = 0
111     while (i < buckets.length) { out += buckets(i); i += 1 }
112     out.toArray
113 }
114
115 /**
116  * Union de tous les buckets, RÉDUITE à m points par le coreset tree.
117  *
118  * C'est le « partial coreset » du §4.1 : ce que chaque subtask parallèle émet
119  * en fin de
120  * flux. Ne vide pas l'état - on peut donc interroger le clustering courant
121  * sans
122  * interrompre la consommation (c'est ce qui rendra possible la variante à
123  * requêtes
124  * périodiques du §4.3, étape E8).
125  */

```

```

123 def extractCoreset(): Array[Point] = CoresetTree.build(union(), m, rng)
124
125 /**
126  * Fusion de deux structures indépendantes, pour le 'combOp' d'un '
127   * treeAggregate' /
128   * 'treeReduce'.
129   *
130   * Repose sur les deux observations du §2.3.4 :
131   * 1. l'union de deux  $\varepsilon$ -coresets d'ensembles disjoints est un  $\varepsilon$ -coreset de l'
132     * union ;
133   * 2. un  $\varepsilon$ -coreset d'un  $\delta$ -coreset est un  $(\varepsilon + \delta + \varepsilon\delta)$ -coreset.
134   * La seconde explique que l'erreur s'accumule à chaque niveau de fusion - c'
135     * est
136   * précisément pourquoi 'streamkm.spark.StreamKMPlusPlus.mergeCoresets'
137     * utilise un
138   * 'treeReduce' de profondeur bornée plutôt qu'une réduction linéaire : la
139     * profondeur de
140   * l'arbre de fusion borne le nombre d'empilements. Voir CLAUDE.md, décision
141     * 11,
142   * pour le calcul explicite de cette borne.
143   *
144   * La réduction 'CoresetTree.build(all, m, out.rng)' utilise le générateur de
145     * 'out' -
146   * celui construit avec la graine combinée 'seed ^ other.seed' - et NON celui
147     * de 'this'
148   * ni celui de 'other'. C'est important : à chaque niveau d'un 'treeReduce',
149     * ce générateur
150   * combine la graine des DEUX opérandes de ce niveau précis, garantissant un
151     * aléa
152   * réellement indépendant à chaque étage de l'arbre de fusion, pas seulement
153     * au premier
154   * niveau (partition → coreset partiel). Utiliser 'this.rng' ou 'other.rng' à
155     * la place
156   * ferait dépendre l'aléa de CE niveau uniquement de l'un des deux opérandes,
157     * en ignorant
158   * l'autre - un bug identifié et corrigé le 2026-08-22 (CLAUDE.md, décision
159     * 10).
160   *
161   * Le résultat est « collapsé » : son état n'est plus un empilement de buckets
162     * valide
163   * au sens des invariants, mais un unique coreset de taille m. Toute insertion
164     * ultérieure
165   * est refusée. C'est sans conséquence pour 'treeAggregate'/'treeReduce', dont
166     * le 'combOp'
167   * ne reçoit jamais que des résultats de 'seqOp'/'combOp', jamais de points
168     * bruts.
169   *
170   * Ne modifie ni 'this' ni 'other'.
171   */
172 def merge(other: BucketManager): BucketManager = {
173   require(other.m == m, s"merge: tailles de coreset incompatibles (${m} vs ${
174     other.m})")
175
176   val out = new BucketManager(m, seed ^ other.seed)
177   val all = union() ++ other.union()
178
179   out.buckets(0).clear()
180   out.ensureLevel(1)
181   out.buckets(1) += CoresetTree.build(all, m, out.rng)
182   out.seen = seen + other.seen
183   out.collapsed = true
184   out
185 }

```

```

167 // -----
168
169
170 private def ensureLevel(level: Int): Unit =
171     while (buckets.length <= level) buckets += new ArrayBuffer[Point](m)
172
173 /**
174  * Vérification des invariants du §2.3.4. Utilisée sous 'assert' : coûteuse et
175  * désactivée en production.
176  */
177 private[coreset] def checkInvariants(): Boolean = {
178     if (buckets(0).length > m) return false
179     var i = 1
180     while (i < buckets.length) {
181         val s = buckets(i).length
182         if (s != 0 && s != m) return false
183         i += 1
184     }
185     true
186 }
187
188 override def toString: String =
189     s"BucketManager(m=$m, seen=$seen, levels=${buckets.map(_.length).mkString("[",
190         ", ", ", ", "]" )})" +
191     (if (collapsed) ", collapsed" else "") + ")"
192 }
193
194 object BucketManager {
195     /**
196      * Taille de coreset recommandée par la thèse (§2.3.4, clarification 1) et
197      * confirmée
198      * expérimentalement au §5.2.1 :  $m = 200 \cdot k$  est un bon compromis temps / qualité.
199      * C'est la valeur par défaut de nos expériences, pas une contrainte de l'
200      * algorithme.
201      */
202     def recommendedM(k: Int): Int = 200 * k
203
204     def apply(m: Int, seed: Long = 42L): BucketManager = new BucketManager(m, seed)
205 }

```

Listing A.5 – streamkm/coreset/BucketManager.scala

A.0.4 Couche spark : portage distribué

```

1 package streamkm.spark
2
3 import org.apache.spark.rdd.RDD
4 import org.apache.spark.sql.DataFrame
5 import org.apache.spark.sql.streaming.StreamingQuery
6
7 import streamkm.core.{KMeans, KMeansModel, Point}
8 import streamkm.coreset.BucketManager
9
10 /**
11  * Paramètres de StreamKM++ (SPEC §4).
12  *
13  * 'm' est la taille de coreset - la thèse recommande  $m = 200 \cdot k$  (§2.3.4,
14  * 'BucketManager.recommendedM'). 'nRestarts' est le nombre d'applications de
15  * k-means++ sur le coreset final (thèse §2.3.4 étape 2 : 5 par défaut ; le

```

```

16  * §5.3.1 montre qu'une seule suffit en pratique - paramètre d'expérience E3,
17  * pas une constante algorithmique).
18  */
19  final case class StreamKMParams(
20      k:          Int,
21      m:          Int,
22      nRestarts: Int = 5,
23      seed:       Long = 42L
24  )
25
26  /**
27   * Couche 3 (SPEC §4) : orchestration Spark de StreamKM++, portage du dataflow
28   * Flink §4.1 de la thèse.
29   *
30   * RÈGLE D'OR (SPEC §1, rappelée ici) : ce fichier ne contient AUCUNE logique
31   * algorithmique. Tout le calcul vient de 'streamkm.core' (KMeans, déjà testé
32   * E1) et 'streamkm.coreset' (BucketManager/CoresetTree, déjà testé E2). Cette
33   * couche se contente de distribuer des appels à ces fonctions sur des
34   * partitions RDD et d'orchestrer le micro-batch Structured Streaming. Aucune
35   * modification n'a été nécessaire dans 'core'/'coreset' pour ça - voir
36   * CLAUDE.md, décision « E3-1 ».
37   */
38  object StreamKMPlusPlus {
39
40      /**
41       * Un coreset partiel par PARTITION - portage direct de l'opérateur
42       * 'FlatMapToPartialCoresets' de la thèse (§4.1) : « each one of the parallel
43       * subtasks... consumes the input points according to algorithm 2.5 ». Une
44       * partition Spark joue exactement le rôle d'un subtask Flink.
45       *
46       * Graine dérivée de l'indice de partition ('p.seed + idx') plutôt que
47       * partagée : nécessaire pour que les expériences de scalabilité (METHODO
48       * §4.0, E3bis) restent reproductibles indépendamment du nombre de
49       * partitions choisi. Avec une graine unique partagée entre partitions,
50       * changer 'numPartitions' changerait aussi la suite aléatoire consommée par
51       * chaque partition, et confondrait l'effet mesuré (le parallélisme) avec un
52       * simple changement de graine - biais qu'on évite ici.
53       */
54      def partialCoresets(rdd: RDD[Point], p: StreamKMParams): RDD[Array[Point]] =
55          rdd.mapPartitionsWithIndex { (idx, it) =>
56              val bm = new BucketManager(p.m, p.seed + idx)
57              bm.insertAll(it)
58              Iterator.single(bm.extractCoreset())
59          }
60
61      /**
62       * Coreset final unique pour tout le RDD - remplace l'opérateur
63       * 'FlatMapToFinalCoreset' de la thèse (§4.1), qui y est **non-parallèle**
64       * (fusion séquentielle des coreset partiels). C'est l'apport principal
65       * identifié par METHODO §3 : la thèse elle-même repère cet opérateur comme
66       * le goulot qui plafonne le speedup au-delà de 8 cœurs (§5.3.1). Ici, la
67       * fusion se fait en arbre via 'RDD.treeReduce'.
68       *
69       * Implémentation : un 'BucketManager' par partition (équivalent d'un
70       * 'seqOp' qui insère tous les points de la partition, déjà couvert par les
71       * tests E2 de 'BucketManager.insert'/'insertAll'), fusionnés deux à deux par
72       * 'BucketManager.merge' (le 'combOp', réutilisé tel quel - voir CLAUDE.md
73       * décision « E3-1 ») via 'treeReduce(f, depth)'.
74       *
75       * Pourquoi pas 'RDD.treeAggregate' directement, alors que la SPEC (§4,
76       * exemple de pipeline) l'utilise avec un 'zeroValue' unique partagé ? Parce
77       * que ce 'zeroValue' unique porte UNE SEULE graine pour toutes les
78       * partitions (Spark clone l'objet par tâche, mais avec le même état RNG

```

```

79  * initial), ce qui casse la recommandation ci-dessus de graines
80  * indépendantes par partition. 'mapPartitionsWithIndex' + 'treeReduce'
81  * produit exactement la même opération (un accumulateur par partition,
82  * fusionnés en arbre de profondeur 'depth') tout en gardant des graines
83  * distinctes. Écart documenté par rapport à l'exemple de code de la SPEC,
84  * mais pas à ses signatures ni à ses invariants.
85  */
86  def mergeCoresets(rdd: RDD[Point], p: StreamKMPParams, depth: Int = 2): Array[
    Point] = {
87      val perPartition: RDD[BucketManager] = rdd.mapPartitionsWithIndex { (idx, it
        ) =>
88          val bm = new BucketManager(p.m, p.seed + idx)
89          bm.insertAll(it)
90          Iterator.single(bm)
91      }
92      perPartition.treeReduce((a, b) => a.merge(b), depth).extractCoreset()
93  }

94
95  /**
96   * Boucle Structured Streaming complète - portage du dataflow entier §4.1 de
97   * la thèse (HdfsSource → FlatMapToPoint → FlatMapToPartialCoresets →
98   * FlatMapToFinalCoreset → FlatMapToKmeansPP → HdfsSink).
99   *
100  * Contrat d'entrée : 'source' doit exposer une colonne 'featuresCol' de type
101  * 'array<double>' (un point brut du flux, poids implicite 1.0 - le poids
102  * n'apparaît jamais dans les données d'entrée, seulement dans les coresets
103  * produits en interne, cf. 'Point.scala'). C'est l'équivalent du
104  * 'FlatMapToPoint' de la thèse, qui parse une ligne en 'Point' ; ici la
105  * conversion est un simple 'Row => Point', volontairement sans passer par un
106  * 'Encoder[Point]' Spark (Point n'est pas un case class, lui en donner un
107  * ajouterait de la complexité sans bénéfice : on repasse en RDD dès la
108  * lecture de la colonne, comme le fait 'mergeCoresets').
109  *
110  * État global : un unique 'BucketManager' maintenu côté driver entre les
111  * micro-batches (mémoire bornée par construction,  $O(m \cdot \log(n/m))$ , thèse
112  * §2.3.4). Chaque micro-batch produit son propre coreset distribué via
113  * 'mergeCoresets', qui est ensuite intégré ('insert') dans cet état global -
114  * exactement la même opération de fusion que 'FlatMapToFinalCoreset',
115  * appliquée cette fois entre micro-batches plutôt qu'entre partitions.
116  *
117  * Instrumentation : une ligne de log par micro-batch (partitions, m, k,
118  * masse vue, temps de construction du coreset, coût du modèle) - le
119  * minimum nécessaire pour l'analyse de scalabilité du ch. 5 du rapport,
120  * sans dépendance à un framework de métriques. 'numPartitions' et 'm' sont
121  * déjà des leviers explicites (le premier via 'source.repartition(N)' avant
122  * l'appel, le second via 'StreamKMPParams.m') : rien à changer ici pour les
123  * faire varier dans les expériences.
124  */
125  def run(
126      source:      DataFrame,
127      p:            StreamKMPParams,
128      onModel:      KMeansModel => Unit,
129      featuresCol:  String = "features"
130  ): StreamingQuery = {
131
132      val global = new BucketManager(p.m, p.seed)
133
134      source.writeStream.foreachBatch { (batch: DataFrame, batchId: Long) =>
135          val t0 = System.nanoTime()
136
137          val points: RDD[Point] = batch.rdd.map { row =>
138              val coords = row.getAs[Seq[Double]](featuresCol).toArray
139              Point(coords)

```

```

140     }
141     val numPartitions = points.getNumPartitions
142
143     val partial = mergeCoresets(points, p.copy(seed = p.seed + batchId))
144     partial.foreach(global.insert)
145
146     val model      = KMeans.fit(global.extractCoreset(), p.k, p.nRestarts, seed
147                               = p.seed)
148     val elapsedMs = (System.nanoTime() - t0) / 1e6
149
150     // scalastyle:off println
151     println(
152       s"[StreamKM++] batch=$batchId partitions=$numPartitions m=${p.m} k=${p.k}
153         } " +
154       f"pointsSeen=${global.numPointsSeen}%.0f buildTimeMs=$elapsedMs%.1f cost
155         =${model.cost}%.4g"
156     )
157     // scalastyle:on println
158
159     onModel(model)
160   }.start()
161 }

```

Listing A.6 – streamkm/spark/StreamKMPlusPlus.scala

```

1 package streamkm.spark
2
3 import org.apache.spark.rdd.RDD
4
5 import streamkm.core.{KMeans, KMeansModel, Point}
6
7 /**
8  * Portage Spark de la méthodologie d'évaluation du coût (thèse §4.2 : job séparé,
9  * indépendant du dataflow §4.1, qui recalcule 'cost(P, C) = Σ_x w_x · D²(x, C)'
10  * en
11  * parallèle sur l'ensemble du flux à partir d'un jeu de centres déjà connu).
12  *
13  * RÈGLE D'OR (SPEC §1) : aucune logique algorithmique ici. La formule du coût
14  * vit dans
15  * 'KMeans.cost' (couche 1, déjà testée par 'KMeansSpec') ; ce fichier ne fait
16  * que la
17  * distribuer sur les partitions d'un RDD et diffuser les centres par broadcast.
18  *
19  * C'est l'instrument de mesure nécessaire à TOUTES les expériences du ch. 5 (E1
20  * -
21  * correction contre une vérité terrain ou MLlib, E2 - trade-off taille de
22  * coreset, E6 -
23  * comparaison de référence) : elles comparent toutes des valeurs produites par
24  * cette
25  * fonction, jamais le coût mesuré sur le coreset lui-même (qui n'est pas
26  * comparable,
27  * cf. le commentaire de 'demo.Main').
28  *
29  * Correspondance avec la thèse (METHODO §3, table "§4.2") :
30  * - 'FlatMapToCentroid' + '.broadcast()' → 'sc.broadcast(centers)'
31  * - 'CoFlatMapToPartialCost' (connected stream) → 'mapPartitions(... KMeans
32  * .cost ...)'
33  * - 'CoFlatMapToFinalCost' (opérateur par. 1) → 'treeReduce(_ + _)'
34  *
35  * Ce que la thèse implémente avec un connected stream, un buffer d'attente et
36  * trois
37  * opérateurs Flink dédiés tient ici en trois lignes de 'broadcast' + '
38  * treeReduce'.

```

```

28  */
29  object CostEvaluator {
30
31    /**
32     * Coût total (SSE pondérée) de 'centers' sur l'ensemble du RDD 'points',
33     * calculé en
34     * parallèle. Chaque partition appelle 'KMeans.cost' (couche 1) sur sa propre
35     * part des
36     * données avec les centres broadcastés, puis les coûts partiels sont sommés
37     * par un
38     * 'treeReduce' (arbre de sommation, plus robuste qu'une réduction linéaire
39     * pour un
40     * grand nombre de partitions - même raisonnement que 'StreamKMPlusPlus.
41     * mergeCoresets').
42     *
43     * Contrairement à la fusion de coresets, cette agrégation est une simple
44     * somme :
45     * aucune composition d'erreur ici (l'addition est associative et exacte à la
46     * précision
47     * flottante près), donc pas de tolérance à dériver - la valeur ne dépend pas
48     * du nombre
49     * de partitions choisi (à l'ordre de sommation flottant près).
50     */
51    def cost(points: RDD[Point], centers: Array[Array[Double]], depth: Int = 2):
52      Double = {
53        val bcCenters = points.sparkContext.broadcast(centers)
54        points
55          .mapPartitions(it => Iterator.single(KMeans.cost(it.toArray, bcCenters.
56            value)))
57          .treeReduce(_ + _, depth)
58      }
59
60    /** Confort : coût d'un 'KMeansModel' déjà ajusté (celui produit par 'onModel'
61     * de 'run'). */
62    def cost(points: RDD[Point], model: KMeansModel): Double = cost(points, model.
63      centers)
64  }

```

Listing A.7 – streamkm/spark/CostEvaluator.scala

A.0.5 Couche experiments : harnais de mesure de qualité

```

1  package streamkm.experiments
2
3  import java.io.{File, FileOutputStream, OutputStreamWriter, PrintWriter}
4  import java.nio.charset.StandardCharsets
5
6  import scala.io.Source
7  import scala.util.Try
8
9  import org.apache.spark.ml.clustering.{KMeans => MLKMeans}
10 import org.apache.spark.ml.linalg.{SQLDataTypes, Vectors}
11 import org.apache.spark.rdd.RDD
12 import org.apache.spark.sql.{DataFrame, Row => SqlRow, SparkSession}
13 import org.apache.spark.sql.types.{StructField, StructType}
14 import org.apache.spark.storage.StorageLevel
15
16 import streamkm.core.{KMeans, Point}
17 import streamkm.coreset.BucketManager
18 import streamkm.spark.{CostEvaluator, StreamKMParams, StreamKMPlusPlus}
19
20

```

```

21 /**
22  * Experiments de qualite de StreamKM++ sur Coverttype.
23  *
24  * Trois approches sont comparees :
25  *
26  * 1. StreamKM++ sequentiel ;
27  * 2. StreamKM++ distribue avec Spark ;
28  * 3. Spark MLlib KMeans comme reference externe.
29  *
30  * La mesure de qualite est toujours la SSE evaluee sur les donnees
31  * originales :
32  *
33  *  $\sum_x \min_c ||x - c||^2$ 
34  *
35  * Le coreset sert uniquement a calculer les centres.
36  */
37 object QualityMetrics {
38
39     // -----
40     // Configuration commune des experiences
41     // -----
42
43     private val Dim = 54
44
45     private val DefaultSeed = 2026L
46     private val DefaultPartitions = 8
47
48     private val NRestarts = 5
49     private val MaxIter = 100
50     private val Tol = 1e-6
51
52     // Valeurs utilisees pour l'etude de l'effet du nombre de clusters.
53     private val KValues =
54         Seq(10, 20, 30, 40, 50)
55
56     // Valeurs utilisees pour l'etude de la taille du coreset.
57     private val MValues =
58         Seq(1000, 5000, 10000, 20000)
59
60
61     final case class ResultRow(
62         dataset: String,
63         method: String,
64         k: Int,
65         m: Int,
66         run: Int,
67         seed: Long,
68         cost: Double
69     )
70
71
72     // =====
73     // Parsing de Coverttype
74     // =====
75
76     /**
77      * Parse une ligne du dataset Coverttype.
78      *
79      * Le fichier UCI contient 55 colonnes :
80      *
81      * - 54 features ;
82      * - 1 label de classe.
83      *

```

```

84     * Comme nous faisons du clustering non supervise, le label n'est pas utilise.
85     */
86     private def parseCovertypeline(
87     line: String,
88     dim: Int
89 ): Option[Point] = {
90
91         val columns = line.trim.split(",")
92
93         if (columns.length < dim) {
94             None
95         } else {
96             Point.parse(
97                 columns.take(dim).mkString(","),
98                 dim
99             )
100         }
101     }
102
103
104     // =====
105     // Lecture locale / sequentielle
106     // =====
107
108     /**
109     * Ouvre Covertypeline et fournit un Iterator[Point].
110     *
111     * Contrairement a l'ancienne implementation, l'ensemble du dataset
112     * n'est PAS charge dans un Array[Point].
113     *
114     * Les points sont lus progressivement depuis le fichier.
115     *
116     * maxPoints est principalement utilise pour les smoke tests :
117     *
118     *   Some(20000) -> premiers 20 000 points
119     *   None        -> dataset complet
120     */
121     private def withPointsFromFile[A](
122     path: String,
123     dim: Int,
124     maxPoints: Option[Int]
125 )(
126     f: Iterator[Point] => A
127 ): A = {
128
129         val source = Source.fromFile(path)
130
131         try {
132
133             val allLines = source.getLines()
134
135             val selectedLines =
136                 maxPoints match {
137                     case Some(n) => allLines.take(n)
138                     case None    => allLines
139                 }
140
141             val points =
142                 selectedLines.flatMap { line =>
143                     parseCovertypeline(line, dim)
144                 }
145
146             f(points)

```

```

147     } finally {
148         source.close()
149     }
150 }
151 }
152
153 /**
154  * Charge une petite partie de Coverttype dans un Array.
155  *
156  * Cette methode est conservee principalement pour les tests de validation
157  * et les petits jeux de donnees.
158  *
159  * Pour les experiences Spark sur le dataset complet, on utilise
160  * loadCoverttypeRDD afin d'eviter de charger toutes les donnees sur le driver.
161  */
162 def loadCoverttype(
163     path: String,
164     dim: Int = Dim,
165     maxPoints: Option[Int] = None
166 ): Array[Point] = {
167
168     withPointsFromFile(
169         path,
170         dim,
171         maxPoints
172     ) { points =>
173         points.toArray
174     }
175 }
176
177 /**
178  * Calcule la SSE d'un ensemble de centres sur un fichier Coverttype
179  * sans charger tout le dataset en memoire.
180  *
181  * Les points sont lus par blocs.
182  *
183  * On reutilise KMeans.cost afin de ne pas dupliquer la formule
184  * mathematique de la SSE.
185  */
186 private def costFromFile(
187     path: String,
188     centers: Array[Array[Double]],
189     dim: Int,
190     maxPoints: Option[Int],
191     chunkSize: Int = 4096
192 ): Double = {
193
194     withPointsFromFile(
195         path,
196         dim,
197         maxPoints
198     ) { points =>
199
200         points
201             .grouped(chunkSize)
202             .map { chunk =>
203                 KMeans.cost(
204                     chunk.toArray,
205                     centers
206                 )
207             }
208             .sum
209     }

```

```

210 }
211
212
213 // =====
214 // Lecture distribuee Spark
215 // =====
216
217 /**
218  * Charge directement Coverttype avec Spark.
219  *
220  * C'est la difference principale avec l'ancienne version :
221  *
222  * AVANT :
223  *
224  *   Source.fromFile
225  *     -> Array[Point]
226  *     -> sc.parallelize(...)
227  *
228  * MAINTENANT :
229  *
230  *   sc.textFile(...)
231  *     -> RDD[String]
232  *     -> RDD[Point]
233  *
234  * Les donnees ne transitent donc plus par un gros Array sur le driver.
235  */
236 def loadCoverttypeRDD(
237   spark: SparkSession,
238   path: String,
239   dim: Int = Dim,
240   numPartitions: Int = DefaultPartitions,
241   maxPoints: Option[Int] = None
242 ): RDD[Point] = {
243
244   val lines =
245     spark.sparkContext.textFile(
246       path,
247       numPartitions
248     )
249
250   /**
251    * Pour le dataset complet, aucune operation supplementaire
252    * n'est necessaire.
253    *
254    * Pour un smoke test avec maxPoints, zipWithIndex permet de
255    * conserver les N premieres lignes sans les collecter sur le driver.
256    */
257   val selectedLines =
258     maxPoints match {
259
260       case Some(n) =>
261         lines
262           .zipWithIndex()
263           .filter {
264             case (_, index) =>
265               index < n.toLong
266           }
267           .keys
268
269       case None =>
270         lines
271     }
272

```

```

273     val parsed =
274         selectedLines.flatMap { line =>
275             parseCovertypeline(line, dim)
276         }
277
278     /*
279     * textFile utilise numPartitions comme minimum.
280     * On garantit ici explicitement le nombre de partitions utilise
281     * par notre protocole experimental.
282     *
283     * Cette repartition n'est faite qu'UNE SEULE FOIS au chargement,
284     * et non a chaque valeur de k ou m.
285     */
286     if (parsed.getNumPartitions == numPartitions) {
287         parsed
288     } else {
289         parsed.repartition(numPartitions)
290     }
291 }
292
293
294 // =====
295 // 1. StreamKM++ sequentiel
296 // =====
297
298 /**
299  * Version de reference sequentielle.
300  *
301  * Etape 1 :
302  * lecture du fichier point par point et construction du coresets.
303  *
304  * Etape 2 :
305  * k-means++ est applique au coresets.
306  *
307  * Etape 3 :
308  * le fichier est relu afin de calculer la SSE sur les donnees originales.
309  *
310  * La memoire utilisee ne depend donc pas directement du nombre total
311  * de points du dataset.
312  */
313 def sequentialCostFromFile(
314     path: String,
315     k: Int,
316     m: Int,
317     seed: Long,
318     dim: Int = Dim,
319     maxPoints: Option[Int] = None
320 ): Double = {
321
322     val manager =
323         new BucketManager(
324             m,
325             seed
326         )
327
328     // Premier passage : construction du coresets.
329     withPointsFromFile(
330         path,
331         dim,
332         maxPoints
333     ) { points =>
334
335         manager.insertAll(points)

```

```

336     }
337
338     require(
339         manager.numPointsSeen > 0.0,
340         s"No valid point found in $path"
341     )
342
343     val coreset =
344         manager.extractCoreset()
345
346     val model =
347         KMeans.fit(
348             coreset,
349             k,
350             nRestarts = NRestarts,
351             maxIter = MaxIter,
352             tol = Tol,
353             seed = seed
354         )
355
356     // Deuxieme passage : cout sur les donnees completes.
357     costFromFile(
358         path,
359         model.centers,
360         dim,
361         maxPoints
362     )
363 }
364
365
366 // =====
367 // 2. StreamKM++ distribue Spark
368 // =====
369
370 /**
371  * Version Spark de StreamKM++.
372  *
373  * Le RDD est deja partitionne et mis en cache dans main.
374  * Il n'est donc pas repartitionne a chaque experience.
375  */
376 def distributedCost(
377     points: RDD[Point],
378     k: Int,
379     m: Int,
380     seed: Long
381 ): Double = {
382
383     val params =
384         StreamKMParams(
385             k = k,
386             m = m,
387             nRestarts = NRestarts,
388             seed = seed
389         )
390
391     val coreset =
392         StreamKMPlusPlus.mergeCoresets(
393             points,
394             params
395         )
396
397     val model =
398         KMeans.fit(

```

```

399         coreset,
400         k,
401         nRestarts = NRestarts,
402         maxIter = MaxIter,
403         tol = Tol,
404         seed = seed
405     )
406
407     /*
408     * Important :
409     * on evalue les centres sur toutes les donnees du RDD,
410     * pas seulement sur le cores et.
411     */
412     CostEvaluator.cost(
413         points,
414         model.centers
415     )
416 }
417
418
419 // =====
420 // 3. Spark MLlib KMeans
421 // =====
422
423 /**
424 * Convertit une seule fois le RDD[Point] en DataFrame MLlib.
425 *
426 * Ce DataFrame sera reutilise pour toutes les valeurs de k.
427 */
428 private def toFeaturesDataFrame(
429     spark: SparkSession,
430     points: RDD[Point]
431 ): DataFrame = {
432
433     val schema =
434         StructType(
435             Seq(
436                 StructField(
437                     "features",
438                     SQLDataTypes.VectorType,
439                     nullable = false
440                 )
441             )
442         )
443
444     val rows =
445         points.map { point =>
446             SqlRow(
447                 Vectors.dense(point.coords)
448             )
449         }
450
451     spark.createDataFrame(
452         rows,
453         schema
454     )
455 }
456
457
458 /**
459 * Reference externe avec Spark MLlib.
460 *
461 * MLlib n'utilise pas notre cores et :

```

```

462  * KMeans est entraine directement sur les donnees completes.
463  *
464  * Pour rendre la comparaison plus stable, plusieurs initialisations
465  * independantes sont testees et le meilleur cout est conserve.
466  *
467  * Remarque :
468  * MLlib utilise son propre mecanisme d'initialisation.
469  * Il s'agit donc d'une reference externe de qualite et non d'une
470  * reproduction exacte de KMeans.fit.
471  */
472  def mllibCost(
473    features: DataFrame,
474    k: Int,
475    seed: Long
476  ): Double = {
477
478    (0 until NRestarts)
479      .map { restart =>
480
481        /*
482         * On espace volontairement les graines entre redemarrages
483         * afin d'eviter que deux repetitions successives utilisent
484         * presque exactement la meme serie de graines.
485         */
486        val restartSeed =
487          seed + restart.toLong * 1000003L
488
489        val model =
490          new MLKMeans()
491            .setK(k)
492            .setSeed(restartSeed)
493            .setFeaturesCol("features")
494            .setMaxIter(MaxIter)
495            .setTol(Tol)
496            .fit(features)
497
498        model.summary.trainingCost
499      }
500    .min
501  }
502
503
504  // =====
505  // Experience A : effet de k
506  // =====
507
508  /**
509   * Fait varier le nombre de clusters k.
510   *
511   * Pour chaque valeur :
512   *
513   *   - m = 200 * k ;
514   *   - StreamKM++ sequentiel ;
515   *   - StreamKM++ Spark ;
516   *   - Spark MLlib.
517   */
518  def runKSweep(
519    spark: SparkSession,
520    path: String,
521    datasetName: String,
522    pointsRDD: RDD[Point],
523    featuresDF: DataFrame,
524    ks: Seq[Int],

```

```

525     baseSeed: Long,
526     nRepeats: Int,
527     maxPoints: Option[Int]
528 ): Seq[ResultRow] = {
529
530     ks.flatMap { k =>
531
532         val m =
533             BucketManager.recommendedM(k)
534
535         (1 to nRepeats).flatMap { run =>
536
537             val seed =
538                 baseSeed + run - 1
539
540             println(
541                 s"[QualityMetrics] k-sweep: " +
542                 s"k=$k m=$m run=$run/$nRepeats seed=$seed"
543             )
544
545             val sequential =
546                 sequentialCostFromFile(
547                     path = path,
548                     k = k,
549                     m = m,
550                     seed = seed,
551                     maxPoints = maxPoints
552                 )
553
554             val distributed =
555                 distributedCost(
556                     pointsRDD,
557                     k,
558                     m,
559                     seed
560                 )
561
562             val mllib =
563                 mllibCost(
564                     featuresDF,
565                     k,
566                     seed
567                 )
568
569             Seq(
570
571                 ResultRow(
572                     datasetName,
573                     "Sequential StreamKM++",
574                     k,
575                     m,
576                     run,
577                     seed,
578                     sequential
579                 ),
580
581                 ResultRow(
582                     datasetName,
583                     "Spark StreamKM++",
584                     k,
585                     m,
586                     run,
587                     seed,

```

```

588         distributed
589     ),
590
591     ResultRow(
592         datasetName,
593         "Spark MLlib KMeans",
594         k,
595         m,
596         run,
597         seed,
598         mllib
599     )
600 )
601 }
602 }
603 }
604
605
606 // =====
607 // Experience B : effet de m
608 // =====
609
610 /**
611  * Fait varier la taille du coreset m avec k fixe.
612  */
613 def runMSweep(
614     path: String,
615     datasetName: String,
616     pointsRDD: RDD[Point],
617     k: Int,
618     ms: Seq[Int],
619     baseSeed: Long,
620     nRepeats: Int,
621     maxPoints: Option[Int]
622 ): Seq[ResultRow] = {
623
624     ms.flatMap { m =>
625
626         (1 to nRepeats).flatMap { run =>
627
628             val seed =
629                 baseSeed + run - 1
630
631             println(
632                 s"[QualityMetrics] m-sweep: " +
633                 s"k=$k m=$m run=$run/$nRepeats seed=$seed"
634             )
635
636             val sequential =
637                 sequentialCostFromFile(
638                     path = path,
639                     k = k,
640                     m = m,
641                     seed = seed,
642                     maxPoints = maxPoints
643                 )
644
645             val distributed =
646                 distributedCost(
647                     pointsRDD,
648                     k,
649                     m,
650                     seed

```

```

651         )
652
653     Seq(
654
655         ResultRow(
656             datasetName,
657             "Sequential StreamKM++",
658             k,
659             m,
660             run,
661             seed,
662             sequential
663         ),
664
665         ResultRow(
666             datasetName,
667             "Spark StreamKM++",
668             k,
669             m,
670             run,
671             seed,
672             distributed
673         )
674     )
675 }
676 }
677 }
678
679
680 // =====
681 // CSV
682 // =====
683
684 /**
685  * Echappe correctement les champs CSV contenant des virgules,
686  * des guillemets ou des retours a la ligne.
687  */
688 private def csvField(
689     value: String
690 ): String = {
691
692     if (
693         value.exists { c =>
694             c == ',' ||
695             c == '"' ||
696             c == '\n' ||
697             c == '\r'
698         }
699     ) {
700
701         "\"" +
702             value.replace("\"", "\\\"") +
703             "\""
704
705     } else {
706         value
707     }
708 }
709
710
711 /**
712  * Ecrit les resultats dans un fichier CSV unique.
713  */

```

```

714 def writeCsv(
715     path: String,
716     rows: Seq[ResultRow]
717 ): Unit = {
718
719     val file =
720         new File(path)
721
722     Option(
723         file.getParentFile
724     ).foreach(
725         _.mkdirs()
726     )
727
728     val writer =
729         new PrintWriter(
730             new OutputStreamWriter(
731                 new FileOutputStream(file),
732                 StandardCharsets.UTF_8
733             )
734         )
735
736     try {
737
738         writer.println(
739             "dataset,method,k,m,run,seed,cost"
740         )
741
742         rows.foreach { row =>
743
744             writer.println(
745                 Seq(
746                     csvField(row.dataset),
747                     csvField(row.method),
748                     row.k.toString,
749                     row.m.toString,
750                     row.run.toString,
751                     row.seed.toString,
752                     row.cost.toString
753                 ).mkString(",")
754             )
755         }
756
757     } finally {
758         writer.close()
759     }
760 }
761
762
763 // =====
764 // Arguments
765 // =====
766
767 /**
768  * Lit un entier positif depuis les arguments.
769  *
770  * Une valeur <= 0 est consideree comme absente.
771  */
772 private def positiveInt(
773     args: Array[String],
774     index: Int
775 ): Option[Int] = {
776

```

```

777     args
778     .lift(index)
779     .flatMap { value =>
780         Try(value.toInt).toOption
781     }
782     .filter(_ > 0)
783 }
784
785
786 // =====
787 // Main
788 // =====
789
790 /**
791  * Utilisation :
792  *
793  * Smoke test 20 000 points :
794  *
795  * sbt "runMain streamkm.experiments.QualityMetrics \
796  * data/covertime/covtype.data 20000 1"
797  *
798  *
799  * Dataset complet, une repetition :
800  *
801  * sbt "runMain streamkm.experiments.QualityMetrics \
802  * data/covertime/covtype.data 0 1"
803  *
804  *
805  * Arguments :
806  *
807  * args(0) : chemin du fichier
808  *
809  * args(1) : nombre maximal de points
810  *           0 = dataset complet
811  *
812  * args(2) : nombre de repetitions
813  */
814 def main(
815     args: Array[String]
816 ): Unit = {
817
818     val path =
819         args.headOption.getOrElse(
820             "data/covertime/covtype.data"
821         )
822
823     val maxPoints =
824         positiveInt(
825             args,
826             1
827         )
828
829     val nRepeats =
830         positiveInt(
831             args,
832             2
833         ).getOrElse(1)
834
835
836     val spark =
837         SparkSession
838             .builder()
839             .appName(

```

```

840     "StreamKM++ Quality Experiments"
841 )
842 .master("local[*]")
843 .config(
844     "spark.ui.enabled",
845     "false"
846 )
847 .config(
848     "spark.sql.shuffle.partitions",
849     DefaultPartitions.toString
850 )
851 .getOrCreate()
852
853
854 // Les logs Spark restent tres bavards.
855 // WARN suffit pour nos experiences locales.
856 spark.sparkContext.setLogLevel("WARN")
857
858
859 try {
860
861     println(
862         s"[QualityMetrics] Loading $path"
863     )
864
865     println(
866         s"[QualityMetrics] maxPoints=$maxPoints"
867     )
868
869     println(
870         s"[QualityMetrics] partitions=$DefaultPartitions"
871     )
872
873
874     // -----
875     // Chargement distribue direct du fichier
876     // -----
877
878     val pointsRDD =
879         loadCovertTypeRDD(
880             spark = spark,
881             path = path,
882             dim = Dim,
883             numPartitions = DefaultPartitions,
884             maxPoints = maxPoints
885         )
886         .persist(
887             StorageLevel.MEMORY_AND_DISK
888         )
889
890
891     try {
892
893         /*
894         * count() force le chargement du RDD.
895         *
896         * A partir de ce moment, les experiences reutilisent les donnees
897         * mises en cache plutot que de relire le fichier a chaque operation
898         * Spark.
899         */
900         val numberOfPoints =
901             pointsRDD.count()

```

```

902
903     require(
904         numberOfPoints > 0,
905         s"No valid point loaded from $path"
906     )
907
908
909     println(
910         s"[QualityMetrics] $numberOfPoints points loaded."
911     )
912
913     println(
914         s"[QualityMetrics] ${pointsRDD.getNumPartitions} Spark partitions."
915     )
916
917
918     // -----
919     // DataFrame MLlib construit une seule fois
920     // -----
921
922     val featuresDF =
923         toFeaturesDataFrame(
924             spark,
925             pointsRDD
926         )
927         .persist(
928             StorageLevel.MEMORY_AND_DISK
929         )
930
931
932     try {
933
934         /*
935          * Force le cache du DataFrame avant les differents appels MLlib.
936          */
937         featuresDF.count()
938
939
940         // =====
941         // Experience k-sweep
942         // =====
943
944         val kSweepRows =
945             runKSweep(
946                 spark = spark,
947                 path = path,
948                 datasetName = "Covertime",
949                 pointsRDD = pointsRDD,
950                 featuresDF = featuresDF,
951                 ks = KValues,
952                 baseSeed = DefaultSeed,
953                 nRepeats = nRepeats,
954                 maxPoints = maxPoints
955             )
956
957
958         writeCsv(
959             "results/quality_k_sweep_covertime.csv",
960             kSweepRows
961         )
962
963
964         // =====

```

```

965 // Experience m-sweep
966 // =====
967
968 val mSweepRows =
969     runMSweep(
970         path = path,
971         datasetName = "Coverttype",
972         pointsRDD = pointsRDD,
973         k = 25,
974         ms = MValues,
975         baseSeed = DefaultSeed,
976         nRepeats = nRepeats,
977         maxPoints = maxPoints
978     )
979
980
981 writeCsv(
982     "results/quality_m_sweep_coverttype.csv",
983     mSweepRows
984 )
985
986
987 println(
988     "[QualityMetrics] Done."
989 )
990
991 println(
992     "  -> results/quality_k_sweep_coverttype.csv"
993 )
994
995 println(
996     "  -> results/quality_m_sweep_coverttype.csv"
997 )
998
999
1000 } finally {
1001
1002     // Libere le cache du DataFrame MLlib.
1003     featuresDF.unpersist()
1004 }
1005
1006
1007 } finally {
1008
1009     // Libere le RDD Coverttype du cache Spark.
1010     pointsRDD.unpersist()
1011 }
1012
1013
1014 } finally {
1015
1016     spark.stop()
1017 }
1018 }
1019 }

```

Listing A.8 – streamkm/experiments/QualityMetrics.scala

A.0.6 Fonction Main

Le main se lance avec :

```
1 sbt run
```

```

1 package streamkm.demo
2
3 import scala.util.Random
4
5 import streamkm.core.{Distance, KMeans, Point}
6 import streamkm.coreset.BucketManager
7
8 /**
9  * Démonstrateur de bout en bout pour les couches E1 ('streamkm.core') et
10 * E2 ('streamkm.coreset'), en l'absence de la couche 'streamkm.spark' (E4,
11 * volontairement hors scope de cette session).
12 *
13 * Rejoue en un seul thread l'algorithme StreamKM++ complet tel que décrit au
14 * §2.3.4 de la thèse (« The StreamKM++ Algorithm ») :
15 * 1. consommer le flux point par point via le merge-and-reduce (algo 2.5) ;
16 * 2. extraire le coreset final (union des buckets non vides, §2.3.4) ;
17 * 3. appliquer k-means++ (seeding  $D^2$  algo 2.2 + Lloyd algo 2.1) sur ce
18 * coreset et garder le meilleur des 'nRestarts' redémarrages - c'est la
19 * même politique que l'opérateur 'FlatMapToKmeansPP' du §4.1 de la thèse.
20 *
21 * C'est exactement le scénario « parallélisme 1 » qui sert de référence à
22 * l'expérience E7 du METHODO (fusion séquentielle vs 'treeAggregate').
23 */
24 object Main {
25
26     /**
27      * Génère un mélange de 'k' gaussiennes isotropes en dimension 'd', à vérité
28      * terrain connue. Les vrais centres sont placés sur une grille de pas
29      * 'spread' pour garantir leur séparabilité - reproduit ici la même
30      * construction que 'TestData.gaussianMixture' (scope test), qu'on ne peut
31      * pas référencer depuis 'src/main'.
32      */
33     private def gaussianMixture(
34         n: Int, k: Int, d: Int, sigma: Double, spread: Double, rng: Random
35     ): (Array[Point], Array[Array[Double]]) = {
36         val trueCenters = Array.tabulate(k) { c =>
37             Array.tabulate(d)(j => ((c + 1) * (j + 1) % k).toDouble * spread)
38         }
39         val points = Array.tabulate(n) { _ =>
40             val ctr = trueCenters(rng.nextInt(k))
41             Point(Array.tabulate(d)(j => ctr(j) + rng.nextGaussian() * sigma))
42         }
43         (points, trueCenters)
44     }
45
46     def main(args: Array[String]): Unit = {
47         val n      = 200000 // taille du flux
48         val k      = 5
49         val d      = 2
50         val m      = BucketManager.recommendedM(k) // 200·k, thèse §2.3.4
51         // clarification 1
52         val seed   = 42L
53         val rng    = new Random(seed)
54
55         val (points, trueCenters) = gaussianMixture(n, k, d, sigma = 0.5, spread =
56             20.0, rng)
57
58         println(s"StreamKM++ (démon E1/E2, sans Spark) - n=$n, k=$k, d=$d, m=$m")
59
60         // --- Étape 1 : consommation du flux, un point à la fois (algo 2.5) ----
61         // Un seul BucketManager = un seul subtask Flink / une seule partition Spark
62         // C'est la structure que E4 fera tourner en parallèle via treeAggregate.

```

```

61    val t0 = System.nanoTime()
62    val manager = BucketManager(m, seed)
63    manager.insertAll(points.iterator)
64    val tIngest = (System.nanoTime() - t0) / 1e6
65
66    // --- Étape 2 : extraction du coreset final (union des buckets, §2.3.4) -
67    val t1      = System.nanoTime()
68    val coreset = manager.extractCoreset()
69    val tExtract = (System.nanoTime() - t1) / 1e6
70
71    // --- Étape 3 : k-means++ sur le coreset (5 redémarrages, §4.1) -----
72    val t2      = System.nanoTime()
73    val model   = KMeans.fit(coreset, k, nRestarts = 5, seed = seed)
74    val tKmeans = (System.nanoTime() - t2) / 1e6
75
76    // Coût mesuré sur les DONNÉES COMPLÈTES avec les centres issus du coreset :
77    // seule comparaison honnête (thèse §4.2,  $\text{cost}(P, C) = \sum D^2(x, C)$ ), le coût
78    // évalué sur le coreset lui-même n'est pas comparable à un coût batch.
79    val costOnFullData = KMeans.cost(points, model.centers)
80    val batchCost      = KMeans.fit(points, k, nRestarts = 5, seed = seed).cost
81    val maxDeviation   = trueCenters.map(tc => math.sqrt(Distance.nearest(tc,
        model.centers)._2)).max
82
83    println(f"Points vus par le manager : ${manager.numPointsSeen}%.0f (attendu
        $n)")
84    println(f"Coreset final           : ${coreset.length} points (m=$m)")
85    println(f"Temps ingestion flux    : $tIngest%.1f ms")
86    println(f"Temps extraction coreset : $tExtract%.1f ms")
87    println(f"Temps k-means++ (coreset) : $tKmeans%.1f ms")
88    println()
89    println(f"Coût (coreset) sur données complètes : $costOnFullData%.2f")
90    println(f"Coût k-means++ batch (référence)      : $batchCost%.2f (ratio ${
        costOnFullData / batchCost}%.3f)")
91    println(f"Écart max centre trouvé / vrai centre : $maxDeviation%.3f")
92    println()
93    println("Centres trouvés :")
94    model.centers.foreach(c => println("  " + c.map(v => f"$v%.2f").mkString(" ",
        ", ", ", ")"))
95  }
96 }

```

Listing A.9 – streamkm/demo/Main.scala : démonstrateur sur données synthétiques

A.0.7 Suite de tests

```

1  package streamkm
2
3  import scala.util.Random
4
5  import streamkm.core.{Distance, Point}
6
7  /**
8   * Générateur de données synthétiques à vérité terrain.
9   *
10  * C'est l'outil de l'expérience E1 (correction) : on connaît les vrais centres,
        donc on
11  * peut vérifier que l'algorithme les retrouve, ce qu'aucun jeu réel ne permet.
        Il servira
12  * aussi en E3/E4 pour faire varier la taille des données à volonté sans dé
        pendre d'un
13  * téléchargement.
14  */

```

```

15 object TestData {
16
17     final case class Mixture(points: Array[Point], trueCenters: Array[Array[Double]
18         ])
19
20     /**
21      * Mélange de 'k' gaussiennes isotropes de dimension 'd', dont les centres
22      * sont placés
23      * sur une grille de pas 'spread' pour garantir la séparabilité.
24      */
25     def gaussianMixture(
26         n:      Int,
27         k:      Int,
28         d:      Int,
29         sigma:  Double = 0.5,
30         spread: Double = 20.0,
31         seed:   Long   = 12345L
32     ): Mixture = {
33         val rng = new Random(seed)
34
35         val centers = Array.tabulate(k) { c =>
36             Array.tabulate(d)(j => ((c + 1) * (j + 1) % k).toDouble * spread)
37         }
38
39         val pts = Array.tabulate(n) { _ =>
40             val c = rng.nextInt(k)
41             val ctr = centers(c)
42             Point(Array.tabulate(d)(j => ctr(j) + rng.nextGaussian() * sigma))
43         }
44
45         Mixture(pts, centers)
46     }
47
48     /** Somme des poids d'un ensemble de points. */
49     def totalMass(pts: Array[Point]): Double = {
50         var s = 0.0
51         var i = 0
52         while (i < pts.length) { s += pts(i).weight; i += 1 }
53         s
54     }
55
56     /**
57      * Distance maximale entre chaque vrai centre et le centre estimé le plus
58      * proche.
59      * Sert à vérifier qu'un clustering a bien « retrouvé » la structure sous-
60      * jacente,
61      * indépendamment de l'ordre des centres.
62      */
63     def maxCenterDeviation(
64         trueCenters: Array[Array[Double]],
65         found:      Array[Array[Double]]
66     ): Double = {
67         var worst = 0.0
68         var i = 0
69         while (i < trueCenters.length) {
70             val d = math.sqrt(Distance.nearest(trueCenters(i), found)._2)
71             if (d > worst) worst = d
72             i += 1
73         }
74         worst
75     }
76 }

```

Listing A.10 – TestData.scala : générateur de données synthétiques

```

1 package streamkm.core
2
3 import scala.util.Random
4
5 import org.scalatest.funsuite.AnyFunSuite
6 import org.scalatest.matchers.should.Matchers
7
8 import streamkm.TestData
9
10 /**
11  * Critères de sortie de l'étape E1.
12  * Tant qu'un de ces tests est rouge, on ne commence pas E2.
13  */
14 class KMeansSpec extends AnyFunSuite with Matchers {
15
16   // -----
17   // Test 1 - valeur analytique connue
18   // -----
19   test("cost : trois points colinéaires, k=1, valeur analytique") {
20     val pts = Array(Point.of(-1.0), Point.of(0.0), Point.of(1.0))
21     // Centre optimal = moyenne = 0 ; SSE = 1 + 0 + 1 = 2
22     KMeans.cost(pts, Array(Array(0.0))) shouldBe 2.0 +- 1e-12
23
24     val model = KMeans.fit(pts, k = 1, nRestarts = 3, seed = 1L)
25     model.centers(0)(0) shouldBe 0.0 +- 1e-9
26     model.cost shouldBe 2.0 +- 1e-9
27   }
28
29   // -----
30   // Test 2 - deux gaussiennes bien séparées
31   // -----
32   test("lloyd : retrouve deux gaussiennes séparées") {
33     val rng = new Random(7L)
34     val pts = Array.tabulate(2000) { i =>
35       val c = if (i % 2 == 0) 0.0 else 10.0
36       Point(Array(c + rng.nextGaussian() * 0.1, c + rng.nextGaussian() * 0.1))
37     }
38
39     val model = KMeans.fit(pts, k = 2, nRestarts = 5, seed = 3L)
40     val dev = TestData.maxCenterDeviation(Array(Array(0.0, 0.0), Array(10.0,
41       10.0)), model.centers)
42     dev should be < 0.05
43   }
44
45   // -----
46   // Test 3 - monotonie du coût
47   // -----
48   test("lloyd : le coût ne remonte jamais") {
49     // L'assertion de monotonie est dans la boucle de 'lloyd' et n'est active
50     // qu'avec -ea (cf. build.sbt). Ce test la déclenche sur une initialisation
51     // volontairement mauvaise, et vérifie en plus la propriété de l'extérieur.
52     val mix = TestData.gaussianMixture(n = 3000, k = 4, d = 3, seed = 99L)
53     val badInit = Array.fill(4)(Array.fill(3)(0.0))
54     val before = KMeans.cost(mix.points, badInit)
55     val model = KMeans.lloyd(mix.points, badInit, maxIter = 50)
56     model.cost should be <= before
57   }
58
59   // -----
60   // Test 4 - pondération (LE test qui attrape le bug classique)
61   // -----
62   test("lloyd : un point de poids w équivaut à w copies du même point") {
63     val a = Array(0.0, 0.0)

```

```

63     val b = Array(10.0, 0.0)
64
65     // Version « répliquée » : 3 copies de a, 1 de b
66     val replicated = Array(Point(a), Point(a), Point(a), Point(b))
67     // Version « pondérée » : a de poids 3, b de poids 1
68     val weighted   = Array(Point(a, 3.0), Point(b, 1.0))
69
70     val init = Array(Array(-5.0, 0.0))
71
72     val mR = KMeans.lloyd(replicated, init, maxIter = 50)
73     val mW = KMeans.lloyd(weighted, init, maxIter = 50)
74
75     // Barycentre attendu : (3·0 + 1·10)/4 = 2.5
76     mR.centers(0)(0) shouldBe 2.5 +- 1e-9
77     mW.centers(0)(0) shouldBe 2.5 +- 1e-9
78     mW.cost shouldBe mR.cost +- 1e-9
79 }
80
81 // -----
82 // Test 5 - seeding
83 // -----
84 test("seedPlusPlus : produit k centres, bornés par n, de la bonne dimension")
85 {
86     val mix = TestData.gaussianMixture(n = 500, k = 5, d = 4, seed = 21L)
87
88     val s = KMeans.seedPlusPlus(mix.points, k = 5, new Random(1L))
89     s.length shouldBe 5
90     s.foreach(_.length shouldBe 4)
91
92     // k > n : on ne peut pas produire plus de centres que de points
93     val tiny = Array(Point.of(0.0), Point.of(1.0))
94     KMeans.seedPlusPlus(tiny, k = 10, new Random(1L)).length shouldBe 2
95 }
96
97 test("seedPlusPlus : le seeding D2 sépare les groupes éloignés") {
98     // 3 amas très distants : sur 20 tirages, le seeding doit quasi toujours
99     // placer un centre dans chaque amas - c'est toute la valeur de k-means++.
100     val rng = new Random(5L)
101     val pts = Array.tabulate(900) { i =>
102         val c = (i % 3) * 1000.0
103         Point(Array(c + rng.nextGaussian()))
104     }
105
106     var successes = 0
107     (0 until 20).foreach { t =>
108         val s      = KMeans.seedPlusPlus(pts, 3, new Random(t.toLong))
109         val groups = s.map(c => math.round(c(0) / 1000.0)).distinct
110         if (groups.length == 3) successes += 1
111     }
112     successes should be >= 18
113 }
114
115 // -----
116 // Test 6 - fit garde bien le meilleur redémarrage
117 // -----
118 test("fit : le coût décroît (au sens large) avec le nombre de redémarrages") {
119     val mix = TestData.gaussianMixture(n = 2000, k = 6, d = 2, sigma = 2.0,
120         seed = 33L)
121     val one  = KMeans.fit(mix.points, k = 6, nRestarts = 1, seed = 8L)
122     val five = KMeans.fit(mix.points, k = 6, nRestarts = 5, seed = 8L)
123     // Même graine : les 5 redémarrages incluent celui de nRestarts=1.
124     five.cost should be <= one.cost + 1e-9
125 }

```

Listing A.11 – core/KMeansSpec.scala

```

1 package streamkm.coreset
2
3 import scala.util.Random
4
5 import org.scalatest.funsuite.AnyFunSuite
6 import org.scalatest.matchers.should.Matchers
7
8 import streamkm.TestData
9 import streamkm.core.{KMeans, Point}
10
11 /**
12  * Critères de sortie de l'étape E2.
13  *
14  * Le test central est la CONSERVATION DE LA MASSE. Un coreset dont la somme des
15  * poids
16  * ne vaut pas n est faux, quelle que soit son apparence - et c'est le seul
17  * symptôme
18  * détectable à coup sûr, parce qu'un coreset légèrement erroné produit tout de
19  * même
20  * des centres plausibles.
21  */
22 class CoresetSpec extends AnyFunSuite with Matchers {
23
24   // =====
25   // CoresetTree
26   // =====
27
28   test("CoresetTree : renvoie l'ensemble tel quel si n <= m") {
29     val pts = TestData.gaussianMixture(n = 50, k = 3, d = 2, seed = 1L).points
30     val cs = CoresetTree.build(pts, m = 100, new Random(1L))
31     cs.length shouldBe 50
32     TestData.totalMass(cs) shouldBe 50.0 +- 1e-9
33   }
34
35   test("CoresetTree : produit au plus m points") {
36     val pts = TestData.gaussianMixture(n = 5000, k = 5, d = 3, seed = 2L).points
37     for (m <- Seq(10, 100, 1000)) {
38       CoresetTree.build(pts, m, new Random(m.toLong)).length should be <= m
39     }
40   }
41
42   test("CoresetTree : conserve exactement la masse totale") {
43     val pts = TestData.gaussianMixture(n = 5000, k = 5, d = 3, seed = 3L).points
44     for (m <- Seq(10, 100, 500)) {
45       val cs = CoresetTree.build(pts, m, new Random(m.toLong))
46       withClue(s"m=$m : ") {
47         TestData.totalMass(cs) shouldBe 5000.0 +- 1e-6
48       }
49     }
50   }
51
52   test("CoresetTree : conserve la masse sur des points déjà pondérés") {
53     val rng = new Random(4L)
54     val pts = Array.tabulate(3000)(_ =>
55       Point(Array(rng.nextGaussian() * 10, rng.nextGaussian() * 10), 1.0 + rng.
56         nextInt(9))
57     )
58     val expected = TestData.totalMass(pts)
59     val cs = CoresetTree.build(pts, m = 200, new Random(5L))
60     TestData.totalMass(cs) shouldBe expected +- 1e-6

```

```

57 }
58
59 test("CoresetTree : gère un ensemble de points tous identiques") {
60   // Coût global nul : la boucle de construction doit s'arrêter sans boucler à
        l'infini.
61   val pts = Array.fill(1000)(Point(Array(3.0, 4.0)))
62   val cs = CoresetTree.build(pts, m = 50, new Random(6L))
63   cs.length should be <= 50
64   TestData.totalMass(cs) shouldBe 1000.0 +- 1e-9
65   cs.foreach { p => p.coords(0) shouldBe 3.0; p.coords(1) shouldBe 4.0 }
66 }
67
68 test("CoresetTree : le coreset préserve la qualité du clustering") {
69   val mix = TestData.gaussianMixture(n = 20000, k = 5, d = 2, sigma = 0.5,
        seed = 7L)
70   val full = KMeans.fit(mix.points, k = 5, nRestarts = 5, seed = 11L)
71
72   val cs = CoresetTree.build(mix.points, m = 200, new Random(12L))
73   val onCoreset = KMeans.fit(cs, k = 5, nRestarts = 5, seed = 11L)
74   // On évalue les centres issus du coreset sur les DONNÉES COMPLÈTES : c'est
        la seule
75   // comparaison honnête (le coût mesuré sur le coreset n'est pas comparable).
76   val costViaCores = KMeans.cost(mix.points, onCoreset.centers)
77
78   withClue(s"complet=${full.cost}, via coreset=$costViaCores : ") {
79     costViaCores should be <= full.cost * 1.10
80   }
81 }
82
83 // =====
84 // BucketManager
85 // =====
86
87 test("BucketManager : compte correctement les points consommés") {
88   val bm = BucketManager(m = 100, seed = 1L)
89   val pts = TestData.gaussianMixture(n = 5000, k = 4, d = 2, seed = 8L).points
90   bm.insertAll(pts)
91   bm.numPointsSeen shouldBe 5000.0 +- 1e-9
92 }
93
94 test("BucketManager : respecte les invariants de taille des buckets") {
95   val bm = BucketManager(m = 64, seed = 2L)
96   val pts = TestData.gaussianMixture(n = 4000, k = 4, d = 2, seed = 9L).points
97   // On vérifie après chaque insertion, pas seulement à la fin.
98   pts.foreach { p =>
99     bm.insert(p)
100     bm.checkInvariants() shouldBe true
101   }
102 }
103
104 test("BucketManager : le nombre de niveaux croît logarithmiquement") {
105   val m = 100
106   val bm = BucketManager(m, seed = 3L)
107   bm.insertAll(TestData.gaussianMixture(n = 100000, k = 3, d = 2, seed = 10L).
        points)
108   //  $O(\log_2(n/m)) + 2 \approx 10 + 2$ . On laisse une marge confortable ; le point du
        test est
109   // qu'on ne soit PAS en  $O(n/m) = 1000$ .
110   bm.numLevels should be <= 16
111 }
112
113 test("BucketManager : union() et extractCoreset() conservent la masse") {
114   val bm = BucketManager(m = 128, seed = 4L)

```

```

115     bm.insertAll(TestData.gaussianMixture(n = 10000, k = 5, d = 3, seed = 11L).
116         points)
117     TestData.totalMass(bm.union())           shouldBe 10000.0 +- 1e-6
118     TestData.totalMass(bm.extractCoreset())  shouldBe 10000.0 +- 1e-6
119     bm.extractCoreset().length               should be <= 128
120 }
121
122 test("BucketManager : extractCoreset ne détruit pas l'état") {
123     val bm = BucketManager(m = 64, seed = 5L)
124     bm.insertAll(TestData.gaussianMixture(n = 5000, k = 3, d = 2, seed = 12L).
125         points)
126
127     val first = bm.extractCoreset()
128     bm.insertAll(TestData.gaussianMixture(n = 5000, k = 3, d = 2, seed = 13L).
129         points)
130     val second = bm.extractCoreset()
131
132     first.length should be <= 64
133     bm.numPointsSeen shouldBe 10000.0 +- 1e-9
134     TestData.totalMass(second) shouldBe 10000.0 +- 1e-6
135 }
136
137 test("BucketManager : merge conserve la masse des deux opérantes") {
138     val a = BucketManager(m = 100, seed = 6L)
139     val b = BucketManager(m = 100, seed = 7L)
140     a.insertAll(TestData.gaussianMixture(n = 4000, k = 4, d = 2, seed = 14L).
141         points)
142     b.insertAll(TestData.gaussianMixture(n = 6000, k = 4, d = 2, seed = 15L).
143         points)
144
145     val merged = a.merge(b)
146     merged.numPointsSeen shouldBe 10000.0 +- 1e-9
147     TestData.totalMass(merged.union()) shouldBe 10000.0 +- 1e-6
148
149     // Les opérantes ne sont pas modifiées (indispensable pour un combOp de
150     // treeAggregate,
151     // que Spark peut ré-exécuter en cas de perte de tâche).
152     a.numPointsSeen shouldBe 4000.0 +- 1e-9
153     b.numPointsSeen shouldBe 6000.0 +- 1e-9
154 }
155
156 test("BucketManager : insertion refusée après merge") {
157     val a = BucketManager(m = 32, seed = 8L)
158     val b = BucketManager(m = 32, seed = 9L)
159     a.insertAll(TestData.gaussianMixture(n = 500, k = 2, d = 2, seed = 16L).
160         points)
161     b.insertAll(TestData.gaussianMixture(n = 500, k = 2, d = 2, seed = 17L).
162         points)
163
164     an[IllegalStateException] should be thrownBy a.merge(b).insert(Point.of(0.0,
165         0.0))
166 }
167
168 // =====
169 // Bout en bout : StreamKM++ séquentiel
170 // =====
171
172 test("StreamKM++ séquentiel : qualité comparable au k-means++ batch") {
173     // C'est l'algorithme complet du §2.3.4, en un seul thread : merge-and-
174     // reduce sur
175     // tout le flux, puis k-means++ sur l'union finale des buckets.
176     // Il servira de référence (« parallélisme 1 ») pour toutes les expériences

```

```

168     de E7.
169     val k      = 5
170     val m      = BucketManager.recommendedM(k) // 200·k, cf. §2.3.4
171     val mix     = TestData.gaussianMixture(n = 50000, k = k, d = 2, sigma = 0.5,
172     seed = 18L)
173
174     val bm      = BucketManager(m, seed = 19L)
175     bm.insertAll(mix.points)
176
177     val finalCoreset = bm.union()
178     val streamModel  = KMeans.fit(finalCoreset, k, nRestarts = 5, seed = 20L)
179     val streamCost    = KMeans.cost(mix.points, streamModel.centers)
180
181     val batchCost    = KMeans.fit(mix.points, k, nRestarts = 5, seed = 20L).cost
182
183     withClue(s"batch=$batchCost, stream=$streamCost : ") {
184         streamCost should be <= batchCost * 1.10
185     }
186     TestData.maxCenterDeviation(mix.trueCenters, streamModel.centers) should be
187         < 0.5
188 }

```

Listing A.12 – coreset/CoresetSpec.scala

```

1 package streamkm.spark
2
3 import org.apache.spark.sql.{SQLContext, SparkSession}
4 import org.apache.spark.sql.execution.streaming.MemoryStream
5 import org.scalatest.BeforeAndAfterAll
6 import org.scalatest.funSuite.AnyFunSuite
7 import org.scalatest.matchers.should.Matchers
8
9 import streamkm.TestData
10 import streamkm.core.{KMeans, KMeansModel}
11 import streamkm.coreset.BucketManager
12
13 /**
14  * Modèle de composition d'erreur pour 'treeReduce' (thèse §2.3.4, observation 2
15  * :
16  * « un  $\varepsilon$ -coreset d'un  $\delta$ -coreset est un  $(\varepsilon+\delta+\varepsilon\delta)$ -coreset », c.-à-d.  $(1+\varepsilon)(1+\delta)-1$ 
17  * -
18  * l'erreur se COMPOSE multiplicativement, pas additivement).
19  *
20  * IMPORTANT - limite honnête : la thèse ne donne aucune formule fermée pour l' $\varepsilon$ 
21  * d'un
22  * coreset produit par 'CoresetTree' (§2.3.4, point 3 : « the authors do not
23  * prove that
24  * the coreset tree structure is indeed a  $(k, \varepsilon)$ -coreset »). Le seul  $\varepsilon$  chiffré de
25  * la thèse
26  * concerne 'AdaptiveCoreset' (théorème 2.1), avec une constante  $O(d)$  non précis
27  * ée - donc
28  * inutilisable numériquement. Ce qu'on PEUT dériver rigoureusement, c'est la
29  * loi de
30  * composition (multiplicative) et le nombre de niveaux de composition qu'
31  * introduit
32  * 'treeReduce'. Pour la magnitude d'un niveau isolé, on réutilise la seule
33  * valeur dont on
34  * dispose : l' $\varepsilon$  empirique déjà validé par 'CoresetSpec' ("le coreset préserve
35  * la qualité
36  * du clustering", 1 seul niveau de réduction, tolérance choisie à 10 % dans ce
37  * repo).
38  */
39 private[spark] object CoresetErrorModel {

```

```

29
30 /**
31  * Facteur de regroupement utilisé par 'RDD.treeReduce'/'treeAggregate' en
    interne
32  * (implémentation Spark : 'scale = max(2, ceil(numPartitions^(1/depth)))').
33  */
34 def scaleFactor(numPartitions: Int, depth: Int): Int =
35     math.max(2, math.ceil(math.pow(numPartitions.toDouble, 1.0 / depth)).toInt)
36
37 /**
38  * Nombre MAXIMUM de fusions 'BucketManager.merge' séquentielles que peut
    subir la
39  * contribution d'un point donné, dans le pire cas, à travers un 'treeReduce'
    de
40  * profondeur 'depth' sur 'numPartitions' partitions.
41  *
42  * Dérivation : à chaque palier (il y en a au plus 'depth'), Spark regroupe
    environ
43  * 'scale' éléments par clé via 'reduceByKey', ce qui replie ce groupe par une
    CHAÎNE de
44  * 'scale - 1' applications séquentielles de 'combOp' (chaque application
    composant à
45  * nouveau l'erreur, cf. observation 2 ci-dessus). D'où la borne 'depth * (
    scale - 1)'.
46  *
47  * Ce n'est PAS une garantie publiée par Spark (treeReduce ne documente pas de
    borne de
48  * composition d'erreur, pour cause : ses 'U' ne sont pas des coresets en géné
    ral) - c'est
49  * une estimation dérivée de sa stratégie de regroupement interne, pour un
    pire cas.
50  */
51 def worstCaseCompositionLevels(numPartitions: Int, depth: Int): Int = {
52     val scale = scaleFactor(numPartitions, depth)
53     depth * (scale - 1)
54 }
55
56 /**  $(1+\epsilon)^{\text{levels}} - 1$  : composition multiplicative d'une erreur  $\epsilon$  sur 'levels'
    niveaux. */
57 def compoundedTolerance(epsPerLevel: Double, levels: Int): Double =
58     math.pow(1.0 + epsPerLevel, levels) - 1.0
59
60 /**
61  * Tolérance relative attendue pour comparer le coût d'un 'mergeCoresets'
    distribué
62  * (sur 'numPartitions' partitions, profondeur 'depth') à une référence sé
    quentielle à un
63  * seul niveau de réduction.
64  *
65  * 'epsPerLevel = 0.10' : repris tel quel de 'CoresetSpec' (E2), qui valide dé
    jà qu'UNE
66  * réduction CoresetTree isolée dégrade le coût de moins de 10 % en pratique
    sur des
67  * mélanges gaussiens comparables. On ne réinvente pas cette calibration ici,
    on la
68  * compose.
69  */
70 def treeReduceTolerance(numPartitions: Int, depth: Int, epsPerLevel: Double =
    0.10): Double =
71     compoundedTolerance(epsPerLevel, worstCaseCompositionLevels(numPartitions,
    depth))
72 }
73

```

```

74 /**
75  * Critères de sortie de l'étape E4.
76  *
77  * Le test central (« non-régression distribué vs séquentiel ») est plus
78  * important ici que la conservation stricte de la masse (déjà couverte par
79  * E2) : il vérifie que distribuer StreamKM++ sur Spark ne dégrade pas la
80  * qualité du clustering par rapport à l'exécution séquentielle de référence
81  * ('demo.Main', ou le test "StreamKM++ séquentiel" de 'CoresetSpec').
82  *
83  * Les tests "état streaming" (fin de fichier) couvrent le risque le plus récent
84  * de la
85  * couche Spark : le double-comptage ou la perte de masse entre micro-batches
86  * successifs
87  * dans l'état 'global' maintenu par 'StreamKMPlusPlus.run'.
88  */
89 class StreamKMSpec extends AnyFunSuite with Matchers with BeforeAndAfterAll {
90
91   @transient private var spark: SparkSession = _
92
93   override def beforeAll(): Unit = {
94     spark = SparkSession.builder()
95       .appName("StreamKMSpec")
96       .master("local[4]")
97       .config("spark.ui.enabled", "false")
98       .config("spark.sql.shuffle.partitions", "4")
99       .getOrCreate()
100   }
101
102   override def afterAll(): Unit = {
103     if (spark != null) spark.stop()
104   }
105
106   // =====
107   // partialCoresets
108   // =====
109
110   test("partialCoresets : un coreset par partition, masse totale conservée") {
111     val mix = TestData.gaussianMixture(n = 8000, k = 4, d = 2, seed = 31L)
112     val rdd = spark.sparkContext.parallelize(mix.points, numSlices = 4)
113
114     val params = StreamKMParams(k = 4, m = 200)
115     val partials = StreamKMPlusPlus.partialCoresets(rdd, params).collect()
116
117     partials.length shouldBe 4
118     partials.foreach(_.length should be <= 200)
119
120     val totalMass = partials.iterator.flatten.map(_.weight).sum
121     totalMass shouldBe 8000.0 +- 1e-6
122   }
123
124   // =====
125   // mergeCoresets
126   // =====
127
128   test("mergeCoresets : conserve la masse totale, quel que soit le nombre de
129     partitions") {
130     val mix = TestData.gaussianMixture(n = 10000, k = 5, d = 2, seed = 32L)
131     val params = StreamKMParams(k = 5, m = BucketManager.recommendedM(5))
132
133     for (nParts <- Seq(1, 2, 8)) {
134       val rdd = spark.sparkContext.parallelize(mix.points, numSlices =
135         nParts)
136       val coreset = StreamKMPlusPlus.mergeCoresets(rdd, params)
137     }
138   }

```

```

133     withClue(s"$nParts=$nParts : ") {
134         coreset.map(_._weight).sum shouldBe 10000.0 +- 1e-6
135         coreset.length should be <= params.m
136     }
137 }
138 }
139
140 // =====
141 // LE TEST LE PLUS IMPORTANT : non-régression distribué vs séquentiel
142 // =====
143
144 test("mergeCoresets sur 8 partitions produit un clustering de qualité
    comparable au séquentiel") {
145     val k = 5
146     val m = BucketManager.recommendedM(k)
147     val mix = TestData.gaussianMixture(n = 40000, k = k, d = 2, sigma =
        0.5, seed = 33L)
148     val numParts = 8
149     val treeDepth = 2 // valeur par défaut de mergeCoresets, non surchargée ici
150
151     // Référence séquentielle : un seul BucketManager, tout le flux d'un coup
152     // (même logique que le test "StreamKM++ séquentiel" de CoresetSpec, E2,
153     // et que demo.Main).
154     val seqBm = new BucketManager(m, seed = 40L)
155     seqBm.insertAll(mix.points)
156     val seqModel = KMeans.fit(seqBm.extractCoreset(), k, nRestarts = 5, seed =
        41L)
157     val seqCost = KMeans.cost(mix.points, seqModel.centers)
158
159     // Version distribuée : mêmes données, réparties sur 'numParts' partitions
160     // Spark.
161     val rdd = spark.sparkContext.parallelize(mix.points, numSlices =
        numParts)
162     val distCoreset = StreamKMPlusPlus.mergeCoresets(rdd, StreamKMParams(k = k,
        m = m, seed = 40L), depth = treeDepth)
163     val distModel = KMeans.fit(distCoreset, k, nRestarts = 5, seed = 41L)
164     val distCost = KMeans.cost(mix.points, distModel.centers)
165
166     // Tolérance DÉRIVÉE (pas codée en dur) de la loi de composition
167     // multiplicative des
168     // coreset (thèse §2.3.4, observation 2) appliquée au nombre de niveaux de
169     // fusion
170     // que 'treeReduce' introduit réellement pour numParts=8, depth=2 (
171     // CoresetErrorModel).
172     // Voir CLAUDE.md, décision 11, pour le calcul détaillé et la comparaison à
173     // l'ancienne valeur codée en dur (25 %).
174     val tolerance = CoresetErrorModel.treeReduceTolerance(numParts, treeDepth)
175
176     withClue(
177         s"séquentiel=$seqCost, distribué=$distCost, tolérance théorique=${
            tolerance * 100}% : "
178     ) {
179         distCost should be <= seqCost * (1.0 + tolerance)
180     }
181     TestData.maxCenterDeviation(mix.trueCenters, distModel.centers) should be <
        2.0
182 }
183
184 test("mergeCoresets : le coût ne se dégrade pas franchement entre 1 et 8
    partitions") {
185     val k = 4
186     val m = BucketManager.recommendedM(k)
187     val mix = TestData.gaussianMixture(n = 20000, k = k, d = 2, sigma = 0.5,

```

```

184         seed = 34L)
185
186     val params = StreamKMParams(k = k, m = m, seed = 50L)
187
188     def costWith(nParts: Int): Double = {
189         val rdd      = spark.sparkContext.parallelize(mix.points, numSlices =
190             nParts)
191         val coresets = StreamKMPlusPlus.mergeCoresets(rdd, params)
192         KMeans.cost(mix.points, KMeans.fit(coresets, k, nRestarts = 5, seed = 51L).
193             centers)
194     }
195
196     val cost1 = costWith(1)
197     val cost8 = costWith(8)
198
199     withClue(s"1 partition=$cost1, 8 partitions=$cost8 : ") {
200         // La thèse observe (§5.3.1, fig. 5.9) que le coût varie peu avec le
201         // parallélisme, et peut même s'améliorer légèrement (plus de listes de
202         // buckets = coresets plus fins). On vérifie ici l'absence de
203         // dégradation franche - même observation que E3bis du METHODODO, en
204         // version test de non-régression plutôt qu'expérience chiffrée.
205         math.abs(cost1 - cost8) should be <= 0.3 * math.max(cost1, cost8)
206     }
207 }
208
209 test("partialCoresets et mergeCoresets utilisent des graines distinctes par
210     partition") {
211     // Les deux fonctions dérivent la graine de chaque partition comme
212     // 'p.seed + idx' (cf. commentaire de StreamKMPlusPlus.partialCoresets).
213     // On vérifie ici que deux partitions contenant les MÊMES points ne
214     // produisent pas des coresets identiques point à point (ce qui trahirait
215     // une graine partagée) : c'est une garantie de non-corrélation, pas une
216     // propriété algorithmique de la thèse, mais elle conditionne la validité
217     // de l'expérience E3bis (isoler l'effet du nombre de partitions de tout
218     // effet de graine).
219     val pts      = TestData.gaussianMixture(n = 2000, k = 3, d = 2, seed = 60L).
220         points
221     val rdd      = spark.sparkContext.parallelize(pts ++ pts, numSlices = 2) // 2
222         copies identiques
223     val params = StreamKMParams(k = 3, m = 50)
224
225     val partials = StreamKMPlusPlus.partialCoresets(rdd, params).collect()
226     partials.length shouldBe 2
227     // Si les deux partitions recevaient exactement les mêmes points originaux
228     // avec la même graine, leurs coresets seraient identiques coordonnée à
229     // coordonnée ; avec des graines distinctes, ce n'est presque sûrement pas
230     // le cas dès qu'un tirage aléatoire intervient (m=50 << n/2=2000, donc le
231     // coresets tree tire bien des échantillons aléatoires).
232     partials(0).map(_.coords.toSeq).toSet should not equal partials(1).map(_.
233         coords.toSeq).toSet
234 }
235
236 // =====
237 // ÉTAT STREAMING DANS LE TEMPS : le point le plus récent et le plus à risque
238 // (double-comptage / perte de masse entre micro-batches successifs)
239 // =====
240
241 test(
242     "état streaming : l'insertion multi micro-batches conserve exactement la
243         masse, " +
244     "sans double-comptage ni perte"
245 ) {
246     // Reproduit exactement la boucle interne de StreamKMPlusPlus.run (

```

```

mergeCoresets par
239 // batch, puis global.insert des représentants), mais avec accès direct à
240 // 'global.numPointsSeen' - ce que le callback 'onModel: KMeansModel => Unit
    ' de 'run'
241 // n'expose pas. C'est le test le plus précis pour la classe de bug visée :
    si un point
242 // était compté deux fois (ou perdu) entre deux micro-batches, '
    numPointsSeen'
243 // décrocherait de la somme cumulée AU MOMENT MÊME où ça se produit - vérifi
    é après
244 // CHAQUE batch, pas seulement à la fin (une compensation entre un batch qui
    perd de la
245 // masse et un autre qui en gagne resterait invisible si on ne vérifiait qu'
    à la fin).
246 val k    = 4
247 val m    = BucketManager.recommendedM(k)
248 val mix = TestData.gaussianMixture(n = 30000, k = k, d = 2, sigma = 0.5,
    seed = 70L)
249
250 val chunks = mix.points.grouped(10000).toArray
251 chunks.length shouldBe 3
252
253 val global = new BucketManager(m, seed = 71L)
254 var cumulativeExpected = 0.0
255
256 chunks.zipWithIndex.foreach { case (chunk, batchId) =>
257     val rdd      = spark.sparkContext.parallelize(chunk, numSlices = 4)
258     val partial = StreamKMPlusPlus.mergeCoresets(rdd, StreamKMParams(k = k, m
        = m, seed = 71L + batchId))
259     partial.foreach(global.insert)
260
261     cumulativeExpected += chunk.length.toDouble
262     withClue(s"après le micro-batch $batchId : ") {
263         global.numPointsSeen shouldBe cumulativeExpected +- 1e-6
264     }
265 }
266
267 TestData.totalMass(global.extractCoreset()) shouldBe 30000.0 +- 1e-6
268 }
269
270 // IGNORE sous Windows sans winutils.exe/HADOOP_HOME : Structured Streaming a
    besoin
271 // d'écrire les métadonnées de checkpoint sur le système de fichiers local, ce
    qui
272 // passe par RawLocalFileSystem.setPermission → Shell.getWinUtilsPath, absent
    de cette
273 // machine (limitation d'environnement Windows connue de Hadoop, sans rapport
    avec le
274 // code de ce projet). Voir CLAUDE.md, décision 13. Fonctionnera tel quel,
    sans aucune
275 // modification, sur Linux/macOS ou dans un conteneur Docker - à réactiver ('
    test' au
276 // lieu d'ignore') dès que l'un de ces environnements est disponible pour
    lancer la
277 // suite.
278 ignore("run() sur plusieurs micro-batches produit une qualité comparable à un
    traitement en un seul bloc") {
279     // Test d'intégration bout-en-bout de 'run', via un MemoryStream Spark pilot
    é
280     // manuellement (addData + processAllAvailable) pour contrôler précisément
    les
281     // frontières de micro-batch - c'est le seul test qui exerce réellement
282     // 'StreamKMPlusPlus.run' (les autres tests appellent 'mergeCoresets'

```

```

    directement).
283 val k      = 4
284 val m      = BucketManager.recommendedM(k)
285 val mix = TestData.gaussianMixture(n = 30000, k = k, d = 2, sigma = 0.5,
    seed = 72L)
286 val chunks = mix.points.grouped(10000).toArray
287 chunks.length shouldBe 3
288
289 val sparkSession: SparkSession = spark // alias stable : 'spark' est un 'var
    ', inutilisable dans un import
290 implicit val sqlCtx: SQLContext = sparkSession.sqlContext
291 import sparkSession.implicit._
292 val source = MemoryStream[Array[Double]]
293 val df      = source.toDF().withColumnRenamed("value", "features")
294
295 val models = scala.collection.mutable.ArrayBuffer.empty[KMeansModel]
296 val query  = StreamKMPlusPlus.run(df, StreamKMParams(k = k, m = m, seed = 73
    L), models += _)
297
298 try {
299     chunks.foreach { chunk =>
300         source.addData(chunk.map(_.coords))
301         query.processAllAvailable()
302     }
303 } finally {
304     query.stop()
305 }
306
307 // Un micro-batch par appel à addData+processAllAvailable : exactement 3 mod
    èles.
308 // Si un batch avait été sauté (perte de masse) ou rejoué (double-comptage),
    ce
309 // nombre resterait 3 mais la qualité ci-dessous décrocherait fortement.
310 models should have length 3
311
312 val streamCost = KMeans.cost(mix.points, models.last.centers)
313 TestData.maxCenterDeviation(mix.trueCenters, models.last.centers) should be
    < 2.0
314
315 // Référence : les mêmes points, en un seul bloc (pas de multi-batch), même
    graine
316 // de driver que le test de masse ci-dessus pour rester comparable.
317 val refBm = new BucketManager(m, seed = 71L)
318 val rdd    = spark.sparkContext.parallelize(mix.points, numSlices = 4)
319 StreamKMPlusPlus.mergeCoresets(rdd, StreamKMParams(k = k, m = m, seed = 71L)
    ).foreach(refBm.insert)
320 val refModel = KMeans.fit(refBm.extractCoreset(), k, nRestarts = 5, seed =
    73L)
321 val refCost  = KMeans.cost(mix.points, refModel.centers)
322
323 // Tolérance : réutilise le même modèle de composition que le test principal
324 // (CoresetErrorModel), en reconnaissant explicitement qu'il est réemployé
    par
325 // commodité plutôt que re-dérivé pour cette composition précise (batch ->
326 // global.insert, différente du treeReduce interne à chaque batch) - voir
327 // CLAUDE.md, décision 12.
328 val tolerance = CoresetErrorModel.treeReduceTolerance(numPartitions = 4,
    depth = 2)
329 withClue(s"streaming (3 micro-batches)=$streamCost, référence (1 bloc)=
    $refCost : ") {
330     streamCost should be <= refCost * (1.0 + tolerance)
331 }
332 }

```

Listing A.13 – spark/StreamKMSpec.scala

```

1 package streamkm.spark
2
3 import org.apache.spark.sql.SparkSession
4 import org.scalatest.BeforeAndAfterAll
5 import org.scalatest.funSuite.AnyFunSuite
6 import org.scalatest.matchers.should.Matchers
7
8 import streamkm.TestData
9 import streamkm.core.KMeans
10
11 /**
12  * Critères de sortie de l'étape E5.
13  *
14  * Contrairement à 'StreamKMSpec' (qui compare des coresets construits avec de l
15  * 'aléa,
16  * donc avec tolérance), le coût est une simple somme : le test central ici est
17  * une
18  * égalité STRICTE (à la précision flottante près) entre le calcul distribué et
19  * le calcul
20  * séquentiel de référence ('KMeans.cost', déjà validé par 'KMeansSpec'). Un é
21  * cart
22  * signifierait un vrai bug de comptage (points oubliés ou comptés deux fois
23  * lors du
24  * partitionnement), pas une variance d'échantillonnage.
25  */
26 class CostEvaluatorSpec extends AnyFunSuite with Matchers with BeforeAndAfterAll
27 {
28
29   @transient private var spark: SparkSession = _
30
31   override def beforeAll(): Unit = {
32     spark = SparkSession.builder()
33       .appName("CostEvaluatorSpec")
34       .master("local[4]")
35       .config("spark.ui.enabled", "false")
36       .getOrCreate()
37   }
38
39   override def afterAll(): Unit = {
40     if (spark != null) spark.stop()
41   }
42
43   test("cost : égal (à la précision flottante près) au calcul séquentiel KMeans.
44     cost") {
45     val mix      = TestData.gaussianMixture(n = 12000, k = 4, d = 3, seed = 80L)
46     val model    = KMeans.fit(mix.points, k = 4, nRestarts = 3, seed = 81L)
47     val refCost  = KMeans.cost(mix.points, model.centers)
48
49     val rdd = spark.sparkContext.parallelize(mix.points, numSlices = 6)
50     val distCost = CostEvaluator.cost(rdd, model.centers)
51
52     // Tolérance relative très fine : seule source d'écart possible = ordre de
53     // sommation
54     // flottant entre partitions, pas un biais d'échantillonnage (cf.
55     // CoresetErrorModel,
56     // sans objet ici puisqu'aucun coreset n'est construit par CostEvaluator).
57     distCost shouldBe refCost +- (refCost.abs * 1e-9 + 1e-9)
58   }
59
60   test("cost : invariant au nombre de partitions") {

```

```

52     val mix    = TestData.gaussianMixture(n = 9000, k = 3, d = 2, seed = 82L)
53     val model = KMeans.fit(mix.points, k = 3, nRestarts = 3, seed = 83L)
54
55     val costs = Seq(1, 3, 9).map { nParts =>
56         val rdd = spark.sparkContext.parallelize(mix.points, numSlices = nParts)
57         nParts -> CostEvaluator.cost(rdd, model.centers)
58     }
59
60     val values = costs.map(_._2)
61     withClue(s"coûts par partitionnement : $costs : ") {
62         values.max - values.min should be <= (values.max.abs * 1e-9 + 1e-9)
63     }
64 }
65
66 test("cost : détecte une régression grossière (mauvais centres = coût très sup
67     érieur)") {
68     // Garde-fou : vérifie que la fonction n'est pas trivialement toujours d'
69     // accord avec
70     // elle-même (un test qui ne peut jamais échouer ne teste rien). Des centres
71     // tirés
72     // loin de la vraie structure doivent produire un coût nettement plus élevé.
73     val mix = TestData.gaussianMixture(n = 6000, k = 4, d = 2, sigma = 0.5, seed
74     = 84L)
75     val goodModel = KMeans.fit(mix.points, k = 4, nRestarts = 5, seed = 85L)
76
77     val badCenters = Array.fill(4)(Array(1000.0, 1000.0)) // loin de toutes les
78     gaussiennes
79
80     val rdd = spark.sparkContext.parallelize(mix.points, numSlices = 4)
81     val goodCost = CostEvaluator.cost(rdd, goodModel.centers)
82     val badCost  = CostEvaluator.cost(rdd, badCenters)
83
84     badCost should be > goodCost * 10.0
85 }
86
87 test("cost : accepte directement un KMeansModel (surcharge de confort)") {
88     val mix    = TestData.gaussianMixture(n = 3000, k = 2, d = 2, seed = 86L)
89     val model = KMeans.fit(mix.points, k = 2, nRestarts = 3, seed = 87L)
90     val rdd    = spark.sparkContext.parallelize(mix.points, numSlices = 3)
91
92     CostEvaluator.cost(rdd, model) shouldBe CostEvaluator.cost(rdd, model.
93     centers)
94 }
95 }

```

Listing A.14 – spark/CostEvaluatorSpec.scala

Bibliographie

- [1] Theodoros Bitsakis. *Clustering Big Data Streams in Apache Flink*. Diploma thesis, 2018.
- [2] Marcel R. Ackermann, Marcus Mörtens, Christiane Raupach, Kamil Swierkot, Christiane Lammersen et Christian Sohler. *StreamKM++ : A clustering algorithm for data streams*. ACM Journal of Experimental Algorithmics (JEA), 17, 2012.
- [3] David Arthur et Sergei Vassilvitskii. *k-means++ : The Advantages of Careful Seeding*. Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms, 2007.